

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

В современных условиях эксплуатации сложных технологических систем крайне важно не только учитывать текущее состояние составляющих систему модулей, но и уметь прогнозировать их поведение на несколько эксплуатационных циклов вперёд. Прогнозирование выхода параметров за допустимые границы позволяет не только снизить риски аварийной эксплуатации, но и повысить общую эффективность производственного процесса, предотвратив экономические издержки, связанные с незапланированными простоями или преждевременным снятием с эксплуатации ещё ресурсных узлов.

Оценка и моделирование текущего состояния оборудования являются ключевыми факторами перехода к предиктивному обслуживанию. Своевременное формирование вероятностных сценариев развития состояния дает возможность использовать оборудование с максимальной эффективностью и оптимизировать графики технического обслуживания. В работах [2–10] рассматривается решение проблем обнаружения аномалий и прогнозирования остаточного ресурса технологического оборудования с использованием моделей глубокого обучения. Особое внимание в современных исследованиях уделяется генеративным архитектурам, способным моделировать распределения временных рядов и синтезировать правдоподобные траектории развития отказов.

Особенностью данной работы является применение вариационного автокодировщика (VAE) в качестве основы системы. Модель решает две сопряженные задачи: классическую задачу восстановления текущих параметров и задачу генерации последующих n эксплуатационных циклов. Амортизация параметров латентного распределения z выполняется не по полному временному ряду, а на основе начальных n циклов, что позволяет эффективно оперировать скалярными показателями состояния St (не являющимися векторами) и генерировать элементы выходной последовательности. Математический аппарат алгоритма базируется на адаптации вариационной нижней оценки (ELBO) под задачу последовательного прогнозирования.

Данная работа имеет практическую значимость в областях производства и эксплуатации технологически сложного оборудования, в частности турбовентиляторных двигателей и промышленных агрегатов. Её применение

позволяет формировать прогнозы будущих показателей циклов, выявлять скрытые закономерности деградации и поддерживать принятие решений в условиях неполных или зашумленных данных.

В связи с этим целью данной работы являлось снижение эксплуатационных рисков и повышение экономической эффективности за счет создания системы моделирования состояния оборудования на базе вариационного автокодировщика, обеспечивающей генерацию последующих циклов и точную оценку технического состояния.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ АНАЛОГОВ

1.1 Обзор исследований и моделей машинного обучения для решения аналогичных задач

1.1.1 Применение классических и глубоких автокодировщиков для оценки состояния оборудования

В работе Jia Guo, Guannan Liu, Yuan Zuo, Junjie Wu под названием “An Anomaly Detection Framework Based on Autoencoder and Nearest Neighbor” [2] исследователи предлагают архитектуру АЕКNN, объединяющую автокодировщик и метод k-ближайших соседей. На этапе обучения модель тренируется исключительно на данных штатного режима. Сжатое представление в латентном слое используется как вход для k-NN классификатора. При тестировании аномалии выявляются по отклонению расстояния до ближайших соседей в скрытом пространстве. Эффективность подхода подтверждается на промышленных датасетах, однако метод не предусматривает генерацию будущих состояний и работает в режиме классификации «штатный/аварийный» без оценки вероятностной неопределенности.

В статье «Обнаружение аномальных событий на хосте с использованием автокодировщика» авторы Гурина А. О., Гузев О. Ю., Елисеев В. Л. предлагают метод построения модели нормального поведения системы на основе двух параллельных автокодировщиков с различной архитектурой. Критерием аномальности выступает превышение порога мгновенной ошибки реконструкции (IRE), рассчитываемого отдельно для каждого декодера. Метод демонстрирует высокую чувствительность к редким событиям, но опирается на детерминированную реконструкцию, что ограничивает его применимость для стохастических процессов износа технологического оборудования.

1.1.2 Вариационные автокодировщики в задачах вероятностного прогнозирования и генерации временных рядов

В работе Kingma and Welling “Auto-Encoding Variational Bayes” [16] заложены теоретические основы VAE, где максимизация вариационной ниж-

ней оценки (ELBO) обеспечивает одновременное обучение кодировщика, декодировщика и регуляризацию латентного пространства. Применительно к телеметрии оборудования данная архитектура позволяет оценивать не только точечное восстановление параметров, но и дисперсию реконструкции, что напрямую связано с надежностью прогноза.

Исследование Chung et al. “Deep Generative Models for Time Series Forecasting” демонстрирует применение рекуррентного VAE (VRNN) для многошагового прогнозирования. Авторы показывают, что аморизованный вывод параметров распределения z по скользящему окну данных позволяет генерировать правдоподобные траектории развития параметров. Однако в работе используется векторное представление состояния на каждом шаге, что увеличивает вычислительную нагрузку и усложняет интерпретацию скалярных индикаторов работоспособности

1.1.3 Сравнительный анализ генеративных архитектур (GAN, Diffusion, VAE) для моделирования циклических процессов

В обзоре Goodfellow et al. по генеративным состязательным сетям и последующих работах по Diffusion-моделям отмечается высокое качество генерации сложных распределений. Однако для задач прогнозирования циклов технологического оборудования эти архитектуры демонстрируют существенные ограничения: GAN страдают от неустойчивости обучения (mode collapse) и отсутствия явной вероятностной интерпретации латентных переменных, что затрудняет калибровку порогов аварийных состояний. Diffusion-модели требуют многоэтапного обратного процесса генерации, что критично увеличивает время инференса и делает их неоптимальными для систем мониторинга в реальном времени. В работе Sohn et al. “Learning Structured Output Representation using Deep Conditional Generative Models” [20] показано, что условные VAE (CVAE) обеспечивают стабильную оптимизацию и явную связь между входным контекстом (текущие n циклов) и выходной последовательностью. Это позволяет формировать прогнозы с оценкой доверительных интервалов, что напрямую соответствует требованиям к системам предиктивного обслуживания.

1.1.4 Адаптация функции ELBO и амортизированный вывод для многошагового прогнозирования состояний

В исследованиях по байесовским методам прогнозирования [21, 22] отмечается, что стандартная формулировка ELBO требует модификации для последовательных данных. Авторы предлагают амортизировать параметры распределения не по полному временному ряду, а по начальному окну из циклов, что снижает риск переобучения и позволяет декодировщику генерировать последующие элементы на основе фиксированного латентного представления. В работе Zheng et al. “Variational Autoencoder for Remaining Useful Life Prediction” скалярный показатель остаточного ресурса выводится как функция от дисперсии реконструкции и расстояния в латентном пространстве. Авторы подчеркивают, что не является вектором признаков, а представляет собой агрегированную метрику, вычисляемую на основе вероятностного вывода. Такой подход позволяет отделить задачу генерации телеметрии от задачи оценки состояния, сохраняя при этом их математическую связность через общий ELBO.

1.2 Критерии оценки и валидации генеративных моделей в промышленной диагностике

Валидация генеративных моделей для временных рядов оборудования требует сочетания поточечных и структурных метрик. В работах [24, 25] базовой метрикой выступает поточечная MSE (Mean Squared Error), вычисляемая для каждого шага прогнозирования: размерность вектора наблюдений. Для учёта фазовых сдвигов и нелинейных деформаций временных осей дополнительно применяются метрики DTW (Dynamic Time Warping) и MAE. В исследовании показано, что для оценки качества генерации будущих циклов недостаточно лишь минимизации MSE, так как модель может «усреднять» распределение. Поэтому вводится критерий коэффициента детерминации по каждому прогнозируемому циклу и F1-score классификации режимов, где порог аномальности определяется на основе распределения ошибок реконструкции и расстояний Махаланобиса в латентном пространстве. Комплексное применение данных метрик позволяет объективно оценить как точность

восстановления, так и устойчивость генерации к зашумленным телеметрическим данным.

1.3 Научная новизна

— Предложен метод прогнозирования остаточного ресурса оборудования на основе вариационного автокодировщика, обучаемого исключительно на данных штатного режима, что позволяет детектировать аномалии и оценивать вероятность отказа без привлечения размеченных выборок аварийных состояний.

— Разработан алгоритм формирования скалярного индикатора риска отказа через вычисление положения ошибки реконструкции в эмпирическом распределении ошибок, построенном на обучающей выборке, обеспечивающий интерпретируемую оценку состояния в диапазоне вероятностей от 0 до 1.

— Обосновано применение механизма амортизированного вывода параметров латентного пространства по скользящему окну из n последовательных циклов, что обеспечивает агрегацию контекста временного ряда, снижение чувствительности модели к высокочастотным шумам телеметрии и повышение устойчивости прогноза к локальным выбросам данных. wide

1.4 Теоретическая и практическая значимость

Теоретическая значимость заключается в обосновании применения вариационного автокодировщика для совместного решения задач многошагового прогнозирования и вероятностной оценки надежности. За счет совместной оптимизации точности генерации будущих циклов и регуляризации скрытого пространства дивергенцией Кульбака-Лейблера обеспечен переход от детерминированного прогнозирования к стохастической оценке состояния, позволяющей количественно измерять неопределенность и риск отказа в условиях отсутствия данных об аварийных режимах.» Полученные результаты создают теоретическую основу для построения систем предиктивного обслуживания, способных количественно оценивать неопределенность прогноза и адаптироваться к изменяющимся условиям эксплуатации без необходимости

переобучения на аварийных сценариях.

Практическая значимость работы заключается в разработке открытого программного комплекса в виде Python-библиотеки, реализующего полный жизненный цикл системы предиктивного обслуживания оборудования. Разработанный инструмент обеспечивает следующее:

1) Обеспечение масштабируемости обработки телеметрических данных. За счет использования генераторов при чтении данных реализована конвейерная обработка потоков информации, что позволяет работать с большими массивами телеметрии без полной выгрузки их в оперативную память. Это критически важно для задач мониторинга парков оборудования, где объемы данных измеряются терабайтами.

2) Ускорение внедрения методов машинного обучения. Модульная структура библиотеки, включающая пакеты предобработки и автоматизированные пайплайны, снижает порог входа для инженеров-разработчиков. Стандартизированные процедуры очистки и нормализации данных минимизируют количество ручных операций при подготовке датасетов.

3) Промышленная готовность и воспроизводимость. Интеграция сервисов MLflow позволяет осуществлять версионирование моделей, логирование гиперпараметров экспериментов и централизованное хранение артефактов. Это обеспечивает прозрачность процесса обучения и возможность быстрого отката к предыдущим версиям моделей при изменении условий эксплуатации.

4) Упрощение процесса принятия решений. В состав библиотеки включены специализированные методы (например, автоматический расчет кумулятивной функции распределения ошибок и прогнозирование остаточного ресурса), которые абстрагируют сложные математические вычисления. Это позволяет эксплуатационному персоналу получать интерпретируемые метрики риска отказа без необходимости глубокого понимания внутреннего устройства нейросетевых архитектур.

5) Возможность независимого использования модулей. Архитектура библиотеки спроектирована таким образом, что каждый пакет (чтение данных, метрики, модели) может быть использован автономно или интегрирован в существующие корпоративные системы мониторинга через программный интерфейс (API).

Валидация разработанных алгоритмов и программной реализации выполнена на базе открытого набора данных NASA C-MAPSS. Полученные результаты подтверждают корректность работы модулей прогнозирования и создают основу для переноса методики на задачи мониторинга реального оборудования.

1.5 Положения, выносимые на защиту

Использование вариационного автокодировщика, обученного на данных нормального функционирования, в сочетании с оценкой эмпирической функции распределения ошибок реконструкции позволяет эффективно детектировать отклонения в работе оборудования и прогнозировать остаточный ресурс с заданной вероятностью, превосходя по устойчивости к шуму методы детерминированного прогнозирования в условиях отсутствия разметки данных об отказах.

1.6 Выводы

В рамках первой главы выполнен анализ предметной области прогнозирования технического состояния сложного оборудования и проведен сравнительный обзор существующих методов машинного обучения. На основе проведенного исследования сформулированы следующие основные результаты:

1) Установлено, что традиционные детерминированные методы прогнозирования не обеспечивают оценки неопределенности прогноза, что критично для задач мониторинга ответственных узлов. Генеративные состязательные сети и диффузионные модели, несмотря на высокую точность генерации, обладают значительной вычислительной сложностью, затрудняющей их применение в системах реального времени.

2) Предложена концепция многошагового прогнозирования на основе амортизированного вывода параметров скрытого распределения. Использование скользящего окна из n предыдущих циклов позволяет модели агрегировать контекст временного ряда, снижая влияние локальных шумов телеметрии на точность оценки текущего состояния.

3) Разработан методологический подход к детектированию аномалий без использования размеченных данных об отказах. Суть подхода заключа-

ется в обучении модели исключительно на данных нормального функционирования и последующем вычислении вероятности отказа через эмпирическую функцию распределения ошибок реконструкции.

4) Сформулированы функциональные и нефункциональные требования к программной реализации системы. Определена модульная архитектура комплекса, включающая конвейер обработки данных, блок генеративного моделирования и сервисы управления экспериментами, что обеспечивает масштабируемость и воспроизводимость результатов. wide

Полученные теоретические и методологические результаты создают необходимую базу для перехода к практической реализации. Во второй главе будет выполнено детальное проектирование системы моделирования, формализована математическая постановка задачи и разработана архитектура программного комплекса.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ МОДЕЛИРОВАНИЯ ДАННЫХ

2.1 Постановка задачи

Для создания системы предиктивного моделирования состояния технологического оборудования в качестве базового источника данных принят датасет NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation), содержащий телеметрические показания турбовентиляторных двигателей в различных режимах эксплуатации и условиях окружающей среды.

Требуется спроектировать и реализовать программный комплекс для вероятностного моделирования и прогнозирования состояния оборудования на основе нейросетевых генеративных моделей. Основными требованиями, предъявляемыми к системе, являются:

- 1) Система должна осуществлять многошаговое прогнозирование телеметрических параметров на n эксплуатационных циклов вперёд с оценкой вероятностной неопределённости.
- 2) Программная реализация должна базироваться на архитектуре вариационного автокодировщика (VAE), обеспечивающей явное вероятностное латентное пространство и устойчивую оптимизацию.
- 3) Система должна принимать на вход временные ряды показаний датчиков в форматах CSV/Parquet. Единицей обработки выступает скользящее окно из n последовательных циклов $X_{1:n}$.
- 4) Система должна формировать скалярный индикатор состояния S_t на основе статистик латентного пространства и ошибки реконструкции, включая векторное представление статуса оборудования.
- 5) Модель должна поддерживать модульную интеграцию: загрузку предобученных весов из удалённого хранилища и экспорт предсказаний в форматы, совместимые с промышленными SCADA/MES-системами.
- 6) Все программные компоненты должны быть оформлены в виде независимых Python-пакетов с чётким разделением ответственности (предобработка, обучение, инференс, валидация).
- 7) В системе должна быть реализована автоматизированная регистрация метрик обучения, гиперпараметров и версий датасетов для обеспечения воспроизводимости экспериментов.

8) Оценка качества модели должна производиться по комплексу метрик: поточечная MSE, DTW для временных рядов, а также классификационные показатели на основе порогов S_t . wide

Входными данными для системы служит телеметрический датасет, отражающий эволюцию технического состояния оборудования во времени. Полный набор данных описывается как $\mathcal{D} = \{\mathbf{X}^{(m)}\}_{m=1}^M$, где M — количество единиц оборудования, а $\mathbf{X}^{(m)} = [\mathbf{x}_1^{(m)}, \mathbf{x}_2^{(m)}, \dots, \mathbf{x}_{T_m}^{(m)}]$ — многомерный временной ряд показаний датчиков для m -го двигателя. Каждый вектор наблюдений $\mathbf{x}_t^{(m)} \in \mathbb{R}^d$ содержит d телеметрических параметров, зарегистрированных в эксплуатационный цикл t .

На этапе обработки данные агрегируются в скользящее окно фиксированной длины n , формирующее входной тензор:

$$X_{1:n} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \in \mathbb{R}^{n \times d} \quad (1)$$

где индекс 1 соответствует началу окна, а индекс n — текущему циклу наблюдения.

Математическое описание выходных данных

Целевым результатом работы системы является генерация вероятностного прогноза будущего состояния оборудования. Выход модели представляет собой последовательность сгенерированных векторов наблюдений для k будущих циклов:

$$\hat{X}_{n+1:n+k} \sim p_\theta(\cdot \mid z), \quad z \sim q_\phi(z \mid X_{1:n}) \quad (2)$$

Процесс формирования выхода состоит из двух этапов:

1) Амортизированный вывод латентного состояния. Кодировщик q_ϕ (нейронная сеть с параметрами ϕ) аппроксимирует апостериорное распределение скрытых переменных на основе входного окна $X_{1:n}$. Из данного распределения сэмплируется латентный вектор $z \in \mathbb{R}^h$, который представляет собой сжатое стохастическое описание текущего технического состояния двигателя с учетом накопленной истории износа.

2) Генерация будущей траектории. Декодировщик p_θ (нейронная сеть с параметрами θ) принимает латентный вектор z в качестве условия и генерирует распределение вероятностей для будущих наблюдений. Математическое

ожидание данного распределения образует матрицу прогноза: wide

$$\hat{X}_{n+1:n+k} = \begin{pmatrix} \hat{\mathbf{x}}_{n+1} \\ \hat{\mathbf{x}}_{n+2} \\ \vdots \\ \hat{\mathbf{x}}_{n+k} \end{pmatrix} \in \mathbb{R}^{k \times d} \quad (3)$$

где $\hat{\mathbf{x}}_t \in \mathbb{R}^d$ — прогнозируемый вектор телеметрии для цикла t , а k — горизонт прогнозирования (количество циклов в будущее).

Таким образом, модель не просто предсказывает точечные значения, а формирует правдоподобную траекторию развития параметров $\hat{X}_{n+1:n+k}$, которая может быть использована для расчета скалярного индикатора состояния S_t и вероятности отказа.

Дополнительно система вычисляет скалярный индикатор состояния $S_t \in \mathbb{R}$ и вероятность отказа $P_{\text{fail}}(t) \in [0, 1]$, определяемые на основе отклонения сгенерированных траекторий от эмпирического распределения ошибок реконструкции, построенного на данных штатного режима.

В рамках данной задачи к обучающим и тестовым наборам предъявляются следующие требования:

- Обучающая выборка должна содержать преимущественно данные, отражающие штатный режим работы оборудования, что позволяет модели выучить распределение нормального состояния и минимизировать ошибку реконструкции.

- Тестовая выборка должна включать последовательности, содержащие как штатные режимы, так и участки деградации или аномальных отклонений, для проверки способности системы к генерации будущих циклов и детектированию изменений состояния.

- Данные должны быть строго разделены на обучающую, валидационную и тестовую выборки без утечки информации между циклами одного двигателя. wide

На первом этапе обработки применяется метод стандартизации данных для приведения разнородных телеметрических каналов к единому масштабу, что критически важно для стабильной оптимизации функции ELBO. Дополнительно выполняется агрегация показаний в окна фиксированной длины n , формирующие входные тензоры $X_{1:n} \in \mathbb{R}^{n \times d}$.

2.2 Математическая формализация генеративной модели

На основе определенных входных данных и целевых переменных формализуем отображение, реализуемое системой. Пусть технологическое оборудование описывается многомерным временным рядом. Формально система должна реализовать отображение $\mathcal{M}$:

$$\mathcal{M} : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{k \times d} \times \mathbb{R}^k \quad (4)$$

где входом является матрица исторического окна $X_{1:n}$, а выходом — пара, состоящая из прогнозируемой последовательности и вектора вероятностных оценок. Данное отображение выполняет две сопряженные функции:

- 1) Восстановление. Реконструкция текущего окна для оценки качества данных и выявления скрытых аномалий;
- 2) Генерация. Формирование последовательности из k будущих циклов с вероятностной оценкой неопределённости. wide

Для реализации отображения $\mathcal{M}$ используется вероятностная модель на базе вариационного автокодировщика. В отличие от детерминированных подходов, модель обучается аппроксимировать апостериорное распределение скрытых переменных, что позволяет учитывать стохастическую природу процессов износа.

Ключевой особенностью подхода является амортизированный вывод параметров латентного распределения z исключительно по начальным n циклам окна. Это позволяет агрегировать контекст до начала прогнозирования, снижая дисперсию скрытых переменных. Параметры распределения z вычисляются кодировщиком с весами ϕ :

$$q_\phi(z \mid X_{1:n}) = \mathcal{N}(z; \mu_\phi(X_{1:n}), \text{diag}(\sigma_\phi^2(X_{1:n}))) \quad (5)$$

Обучение модели проводится путём максимизации вариационной нижней оценки (ELBO), адаптированной для задачи условного многошагового прогнозирования:

$$\mathcal{L}(\theta, \phi; X_{1:n+k}) = \mathbb{E}_{q_\phi(z \mid X_{1:n})} [\log p_\theta(X_{n+1:n+k} \mid z)] - \lambda \cdot D_{KL}(q_\phi(z \mid X_{1:n}) \parallel p(z)) \quad (6)$$

где $p_\theta(X_{n+1:n+k} \mid z)$ — вероятностная модель декодировщика, генерирующая последующие циклы; $p(z) = \mathcal{N}(0, I)$ — априорное стандартное нормальное распределение; λ — коэффициент регуляризации; D_{KL} — дивергенция Кульбака-Лейблера, обеспечивающая структурированность латентного пространства.

Для количественной оценки состояния вводится скалярный индикатор S_t , вычисляемый как нормированная комбинация ошибки реконструкции и расстояния в латентном пространстве:

$$S_t = \frac{1}{k} \sum_{i=n+1}^{n+k} (w_1 \cdot \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2 + w_2 \cdot D_{KL}(q_\phi(z \| X_{1:n}) \| p(z))) \quad (7)$$

где w_1, w_2 — эмпирические весовые коэффициенты. Поскольку $S_t \in \mathbb{R}$, он позволяет порогово разделять режимы работы на три класса: норма, предупреждение, критическое состояние.

Таким образом, математический аппарат определяет входные данные, выходные последовательности и механизм их совместного моделирования в рамках единой генеративной архитектуры.

2.3 Архитектура нейросетевой модели и методы валидации

На основе формализованного отображения требуется определить структуру кодировщика и декодировщика, способную эффективно обрабатывать многомерные временные ряды телеметрии. В данном разделе проводится сравнительный анализ архитектурных решений, обосновывается выбор компонентов модели и формируются критерии оценки качества.

2.3.1 Сравнительный анализ архитектур кодировщика и декодировщика

Для задач последовательного прогнозирования и генерации временных рядов применяются четыре основных класса архитектур, интегрируемых в схему вариационного автокодировщика:

Многоуровневые персептроны (MLP-VAE). Полносвязные слои обрабатывают каждое окно данных как независимый вектор фиксированной размерности. Преимуществом является низкая вычислительная сложность и простота реализации. Недостатком выступает отсутствие явного учета вре-

менной зависимости между циклами внутри окна, что снижает точность долгосрочного прогнозирования и увеличивает чувствительность к фазовым сдвигам телеметрии.

Сверточные архитектуры (CNN-VAE). Одномерные сверточные слои извлекают локальные паттерны и инвариантные признаки временного ряда. Метод эффективен для выявления краткосрочных аномалий и шумовых выбросов. Однако свертки слабо моделируют долговременные зависимости и требуют значительного объема памяти для больших горизонтов прогнозирования.

Рекуррентные архитектуры (RNN-VAE, LSTM-VAE, GRU-VAE). Рекуррентные блоки последовательно обрабатывают элементы окна, сохраняя скрытое состояние, которое агрегирует историю наблюдений. Данная архитектура обеспечивает явное моделирование временных зависимостей, устойчива к пропуску циклов и демонстрирует высокую точность при многошаговом прогнозировании. Вычислительная нагрузка умеренная, что позволяет проводить обучение и инференс на стандартном оборудовании.

Трансформерные архитектуры (Transformer-VAE). Механизм внимания позволяет моделировать глобальные зависимости во всем окне без рекуррентных ограничений. Метод показывает наилучшие результаты на длинных последовательностях, но требует значительных вычислительных ресурсов и больших объемов данных для стабилизации обучения, что ограничивает его применение в системах мониторинга с жесткими требованиями к времени отклика.

На основе сравнительного анализа для реализации системы выбран вариант на базе рекуррентных блоков (GRU/LSTM). Данная архитектура обеспечивает баланс между точностью учета временной динамики, вычислительной эффективностью и устойчивостью к шумам телеметрии. Кодировщик формирует параметры распределения $q_{\phi}hi(z|X)$ на основе последнего скрытого состояния рекуррентной сети, а декодировщик разворачивает латентный вектор в последовательность прогнозируемых циклов с использованием рекуррентной генерации или прямого многошагового вывода.

2.4 Метрики оценки качества моделирования

Оценка результатов работы модели проводится по трем направлениям: точность прогнозирования, качество детектирования аномалий и вероятностная согласованность.

Для оценки точности генерации будущих циклов применяются поточечные метрики регрессии: Среднеквадратичная ошибка (RMSE) характеризует среднеквадратичное отклонение прогнозируемых значений от эталонных и чувствительна к крупным выбросам. Средняя абсолютная ошибка (MAE) измеряет среднее абсолютное отклонение и обладает устойчивостью к редким аномальным пикам. Коэффициент детерминации (R^2) отражает долю дисперсии телеметрических параметров, объясненную моделью.

Для учета временной структуры данных применяется метрика динамической трансформации времени (DTW). Данный критерий измеряет минимальное расстояние между двумя последовательностями с учетом нелинейных фазовых сдвигов, что позволяет корректно сравнивать траектории с различной скоростью деградации.

Для оценки качества детектирования аномалий и классификации состояний используются: F1-мера, вычисляемая как гармоническое среднее точности и полноты на основе порогового значения скалярного индикатора S_t . Площадь под ROC-кривой (AUC-ROC), характеризующая способность модели разделять штатные и аномальные режимы независимо от выбора порога. Специализированная функция потерь NASA Score, применяемая для оценки прогнозирования остаточного ресурса. Метрика асимметрично штрафует ошибки: запоздалые прогнозы отказа получают больший штраф, чем преждевременные, что соответствует требованиям безопасности эксплуатации.

Для оценки вероятностной согласованности генеративной модели контролируются: Значение вариационной нижней оценки (ELBO) на валидационной выборке. Отрицательное правдоподобие (NLL), вычисляемое на основе параметров выходного распределения декодировщика. Среднее значение дивергенции Кульбака-Лейблера, характеризующее степень регуляризации латентного пространства.

2.5 Методы статистического сравнения результатов

Сравнение архитектурных решений и оценка значимости различий в работе системы проводятся с применением методов статистического анализа, обеспечивающих воспроизводимость и надежность выводов.

Все эксперименты выполняются с фиксацией случайного начального состояния (seed) и повторяются не менее пяти раз для каждой конфигурации. Итоговые показатели усредняются, а разброс результатов характеризуется стандартным отклонением и доверительным интервалом с уровнем значимости 0.95.

Для попарного сравнения двух архитектур на одних и тех же тестовых двигателях применяется непараметрический критерий Уилкоксона для связанных выборок. Данный метод не требует предположения о нормальном распределении метрик и устойчив к выбросам, что характерно для результатов обучения нейросетевых моделей.

При одновременном сравнении трех и более архитектур используется критерий Фридмана для многократных измерений. При обнаружении статистически значимых различий выполняется апостериорное сравнение по методу Нимейни для выявления конкретных пар архитектур, различающихся по качеству.

Для оценки практической значимости различий вычисляется размер эффекта (коэффициент Коэна d). Значения размера эффекта интерпретируются как малые, средние и большие в зависимости от степени перекрытия распределений метрик сравниваемых моделей.

Дополнительно проводится анализ устойчивости к изменению гиперпараметров и объема обучающей выборки. Строятся кривые обучения и валидации, фиксируется момент сходимости функции потерь и отсутствие переобучения. Статистическая значимость различий в скорости сходимости и стабильности обучения проверяется с применением критерия Краскела-Уоллиса.

Комплексное применение указанных метрик и статистических процедур обеспечивает объективное сравнение архитектурных решений, подтверждает преимущества выбранной конфигурации и формирует доказательную базу для перехода к программной реализации и экспериментальному исследованию.

2.6 Этапы проведения экспериментов и проверки гипотезы

Проверка сформулированной гипотезы осуществляется в рамках многоэтапного экспериментального плана. Процедура включает подготовку данных, обучение генеративной модели, калибровку порогов аномальности и итоговую валидацию системы прогнозирования остаточного ресурса.

2.6.1 Подготовка данных и формирование выборок

Для обеспечения корректности эксперимента исходный датасет NASA C-MAPSS разделяется на две независимые части: обучающую и тестовую.

Обучающая выборка формируется исключительно из данных штатного режима работы двигателей. Для этого отбираются первые n циклов каждого двигателя, где процесс деградации еще не проявился в телеметрических показателях. Данные подвергаются стандартизации (Z-score normalization) для приведения разнородных датчиков к единому масштабу. После нормализации временные ряды агрегируются в скользящие окна фиксированной длины n , формируя тензоры вида $X_{1:n}$.

Тестовая выборка включает полные траектории двигателей, содержащие как штатные режимы, так и участки прогрессирующего износа вплоть до отказа. Данный набор используется для оценки способности модели детектировать аномалии и прогнозировать остаточный ресурс на данных, которые не использовались при обучении.

2.6.2 Обучение модели и оптимизация гиперпараметров

Обучение вариационного автокодировщика проводится путем минимизации функции потерь ELBO на обучающей выборке. Процесс оптимизации осуществляется с использованием стохастического градиентного спуска (Adam).

На данном этапе решается задача подбора оптимальных гиперпараметров архитектуры: Размерности латентного пространства z . Длины исторического окна n . Горизонта прогнозирования k .

Поиск оптимальных значений проводится методом перебора по сетке (Grid Search) с валидацией на отложенном подмножестве обучающих данных. Критерием остановки обучения служит стабилизация значения функции потерь на валидационном множестве в течение определенного количества эпох

(Early Stopping), что предотвращает переобучение модели на шумовые компоненты телеметрии.

2.6.3 Инференс и калибровка порога детектирования

После завершения обучения модель применяется к тестовой выборке в режиме инференса. Для каждого окна $X_{1:n}$ вычисляются параметры апостериорного распределения и генерируется прогнозируемая последовательность.

На основе полученных результатов вычисляется скалярный индикатор состояния S_t для каждого цикла тестовой выборки. Индикатор рассчитывается как взвешенная сумма ошибки реконструкции и дивергенции Кульбака-Лейблера.

Ключевым этапом является определение граничного порога аномальности. Для этого строится эмпирическая функция распределения (CDF) значений индикатора S_t , полученных на обучающей выборке (данные нормы). Пороговое значение `threshold` выбирается таким образом, чтобы доля ложных срабатываний (False Positive Rate) не превышала заданного уровня доверия (например, 95

2.6.4 Оценка качества прогнозирования и статистическая валидация

Финальный этап эксперимента направлен на количественную оценку эффективности системы и проверку статистической значимости полученных результатов.

Для оценки точности прогноза вычисляются метрики RMSE, MAE и DTW между сгенерированными траекториями и реальными показаниями датчиков на тестовой выборке.

Для оценки способности системы к прогнозированию остаточного ресурса (RUL) применяется функция потерь NASA Score, которая учитывает асимметричную стоимость ошибок прогнозирования.

Статистическая проверка гипотезы о преимуществе выбранной архитектуры VAE перед базовыми моделями (Standard AE, LSTM) проводится с использованием непараметрического критерия Уилкоксона для связанных выборок. Сравнение выполняется по основным метрикам качества (RMSE, Score) на одних и тех же тестовых двигателях.

Результатом экспериментального исследования является подтвержде-

ние того, что вероятностная модель на базе VAE обеспечивает более высокую устойчивость к шуму и точность прогнозирования состояния оборудования по сравнению с детерминированными аналогами, а также позволяет корректно оценивать вероятность отказа на основе скалярного индикатора S_t .

2.7 Концептуальное проектирование системы

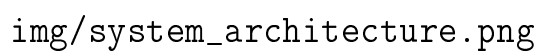
На основе сформулированной постановки задачи и математического аппарата вариационного автокодировщика разработана концептуальная схема программного комплекса. Система проектируется как модульная структура, реализующая полный жизненный цикл предиктивного обслуживания: от приема сырых телеметрических сигналов до формирования вероятностных прогнозов и логирования экспериментов.

Архитектура комплекса разделена на функциональные пакеты, каждый из которых отвечает за определенный этап обработки данных. Пакет чтения данных реализует поточную загрузку временных рядов с использованием генераторов, что исключает полную выгрузку массивов в оперативную память. Пакет предобработки выполняет фильтрацию высокочастотных шумов, интерполяцию пропусков и стандартизацию каналов. Пакет пайплайна аккумулирует методы трансформации данных и формирует скользящие окна фиксированной длины $X_{1:n}$.

Пакет моделей содержит реализацию архитектуры VAE, включая процедуры амортизированного вывода параметров латентного распределения, генерации будущих последовательностей $\hat{X}_{n+1:n+k}$ и вычисления скалярного индикатора состояния S_t . Пакет метрик обеспечивает расчет поточечных ошибок, структурных показателей временных рядов и специализированных функций потерь для оценки остаточного ресурса. Сервисы интеграции с платформой управления экспериментами отвечают за версионирование артефактов, логирование гиперпараметров и централизованное хранение обученных весов.

Взаимодействие компонентов и потоки данных представлены на рисунке ??.

Как отражено на рисунке ??, поток обработки начинается с приема сырых показаний датчиков. Данные проходят через конвейер нормализации и

The image is a large, empty rectangular box with a thin black border. Inside the box, the text 'img/system_architecture.png' is written in a monospaced font, indicating that the actual diagram is missing from the page.

img/system_architecture.png

Рисунок 1 — Система, обеспечивающая работу с данными и моделями для решения поставленной задачи

агрегации, после чего сохраняются в структурированном виде. Модуль обучения извлекает подготовленные окна, оптимизирует функцию ELBO и калибрует пороги аномальности на основе эмпирического распределения ошибок. Готовая модель экспортируется в удаленное хранилище, откуда может быть загружена для инференса.

В рабочем режиме система принимает новое окно наблюдений, выполняет прямой проход через кодировщик, генерирует прогноз и вычисляет вероятность риска отказа. Результаты мониторинга вместе с метриками качества передаются в подсистему логирования, обеспечивая прозрачность процесса принятия решений и возможность ретроспективного анализа.

Подобная модульная организация обеспечивает гибкость масштабирования, независимое обновление компонентов и полное соответствие требованиям к воспроизводимости исследований. Разработанная концепция служит основой для программной реализации, описание которой будет приведено в следующей главе.

Выводы

В рамках второй главы выполнено детальное проектирование системы моделирования состояния технологического оборудования и сформирована ее математическая основа. Сформулированы следующие основные результаты:

1) Сформирована формальная постановка задачи, определяющая входные данные в виде скользящего окна $X_{1:n}$ и целевую переменную как генеративную последовательность с вероятностной оценкой. Введен скалярный индикатор состояния S_t , вычисляемый на основе ошибки реконструкции и дивергенции Кульбака-Лейблера для количественного измерения риска отказа.

1) Сформирована формальная постановка задачи, определяющая входные данные в виде скользящего окна $X_{1:n}$ и целевую переменную как генеративную последовательность с вероятностной оценкой. Введен скалярный индикатор состояния S_t , вычисляемый на основе ошибки реконструкции и дивергенции Кульбака-Лейблера для количественного измерения риска отказа.

2) Обоснован выбор архитектуры вариационного автокодировщика

(VAE) в качестве базового инструмента. Показано, что совместная оптимизация точности генерации и регуляризации скрытого пространства позволяет детектировать аномалии без использования размеченных данных об отказах, что является ключевым преимуществом подхода для реальных промышленных условий.

3) Разработана концептуальная архитектура программного комплекса, реализующая модульную структуру с разделением на пакеты чтения данных, предобработки, моделирования, оценки метрик и логирования экспериментов. Архитектура обеспечивает конвейерную обработку потоков данных и масштабируемость системы.

4) Определены критерии оценки качества моделирования и план экспериментальной проверки гипотезы. Выбран набор метрик (RMSE, DTW, NASA Score) и методы статистической валидации (критерий Уилкоксона) для объективного сравнения разработанного подхода с аналогами.

Полученные результаты проектирования создают необходимую теоретическую и архитектурную основу для перехода к программной реализации и экспериментальному исследованию, которые будут рассмотрены в четвертой главе.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ МОДЕЛИРОВАНИЯ

3.1 Архитектура программного комплекса

Архитектура программного комплекса должна представлять собой модульную систему, реализующую полный жизненный цикл предиктивного обслуживания технологического оборудования. Архитектура построена по принципу разделения ответственности и включает четыре функциональных слоя: слой данных, слой предобработки, слой моделирования и слой метрик. Интеграция с платформой MLflow обеспечивает версионирование артефактов и воспроизводимость экспериментов.

Слой данных отвечает за хранение и управление входными потоками телеметрии. Он включает хранилище сырых данных, содержащих показания датчиков в исходном формате, и хранилище предобработанных данных, куда записываются нормализованные временные ряды, готовые к обучению. Разделение на два хранилища позволяет избежать повторной обработки данных при многократных запусках экспериментов и ускоряет итерации разработки моделей.

Слой предобработки реализует конвейер трансформации данных. Модуль загрузки данных (Data Loader) обеспечивает потоковое чтение файлов без полной выгрузки в оперативную память, что критически важно для работы с большими массивами телеметрии. Модуль очистки (Data Cleaner) выполняет интерполяцию пропущенных значений и фильтрацию высокочастотных шумов. Модуль нормализации (Normalizer) применяет Z-score стандартизацию для приведения разнородных датчиков к единому масштабу. Генератор окон (Window Generator) агрегирует последовательные циклы в тензоры фиксированной размерности $X_{1:n}$, формируя входные данные для обучения.

Слой моделирования содержит ядро системы — вариационный автокодировщик. Кодировщик (Encoder) реализует амортизированный вывод параметров латентного распределения $q_\phi(z \mid X_{1:n}) = \mathcal{N}(z; \mu_\phi, \text{diag}(\sigma_\phi^2))$. Латентное пространство (Latent Space) обеспечивает хранение сжатого стохастического представления состояния оборудования. Декодировщик (Decoder) генерирует последовательность будущих циклов $\hat{X}_{n+1:n+k}$ на основе сэмплированного вектора z . Детектор аномалий (Anomaly Detector) вычисляет скалярный индикатор состояния S_t и вероятность отказа на основе эмпирической

функции распределения ошибок реконструкции.

Слой метрик отвечает за количественную оценку качества моделирования. Калькулятор метрик (Metrics Calculator) вычисляет поточечные ошибки (RMSE, MAE) и структурные показатели (DTW) для сгенерированных траекторий. Калибратор порогов (Threshold Calibrator) определяет граничные значения индикатора S_t на основе распределения ошибок на обучающей выборке. Предиктор остаточного ресурса (RUL Predictor) экстраполирует тренд индикатора состояния и оценивает время до пересечения критического порога.

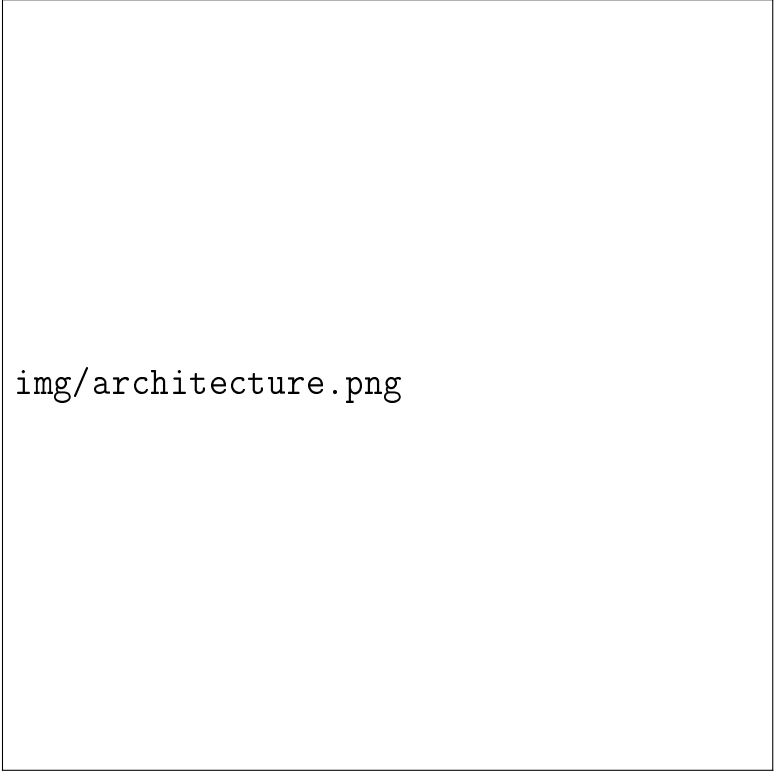
Интеграция с MLflow реализована через два компонента. Реестр моделей (Model Registry) хранит веса обученных нейросетей, параметры калибровки порогов и метаданные экспериментов. Трекер экспериментов (Exp Tracker) фиксирует значения метрик, гиперпараметры и версии данных, обеспечивая полную воспроизводимость исследований.

Архитектура поддерживает три основных сценария функционирования. Сценарий обучения (обозначен сплошными линиями на рисунке) включает загрузку сырых данных, их предобработку, оптимизацию функции ELBO и сохранение модели в реестр. Сценарий инференса (пунктирные линии) выполняет загрузку новой порции телеметрии, генерацию прогноза и вычисление вероятности отказа с использованием сохранённой модели. Сценарий дообучения (штрихпунктирные линии) позволяет адаптировать модель к изменяющимся условиям эксплуатации путём тонкой настройки весов на новых данных без полного переобучения.

Подобная модульная организация обеспечивает гибкость масштабирования, независимое обновление компонентов и возможность интеграции с промышленными системами мониторинга. Детальная структура программных пакетов и описание их взаимодействия будут приведены в следующих разделах.

3.2 Структура Python-библиотеки и организация пакетов

Программный комплекс реализован в виде открытой Python-библиотеки с модульной архитектурой, обеспечивающей четкое разделение ответственности между компонентами. Структура проекта построена по принципу



img/architecture.png

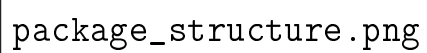
Рисунок 2 — Архитектура системы обучения и прогнозирования

независимых пакетов, что позволяет использовать каждый модуль автономно или в составе единого конвейера обработки данных. Корневая директория библиотеки содержит конфигурационные файлы, документацию и исходный код, организованный в следующие пакеты:

Пакет данных отвечает за загрузку телеметрических рядов из внешних источников. Реализация основана на использовании итераторов и генераторов, что обеспечивает конвейерную обработку потоков информации без полной выгрузки массивов в оперативную память. Поддерживаются форматы CSV и Parquet, а также механизмы кэширования часто используемых сегментов временных рядов.

Пакет предобработки содержит процедуры фильтрации высокочастотных шумов, интерполяции пропущенных значений и синхронизации временных меток. Модуль нормализации применяет Z-score стандартизацию на основе статистик обучающей выборки, что гарантирует согласованность масштабов разнородных датчиков. Генератор скользящих окон преобразует непрерывные временные ряды в тензоры фиксированной размерности $X_{1:n}$, готовые для подачи в нейросетевую архитектуру.

Пакет пайплайна выполняет функцию оркестратора, координируя по-

A diagram illustrating the structure of Python packages, showing a hierarchy of directories and files. The diagram is not visible in the provided image, but the filename 'package_structure.png' is present.

package_structure.png

Рисунок 3 — Структура пакетов Python-библиотеки

последовательность вызовов модулей чтения, трансформации и обучения. Конфигурационный менеджер позволяет задавать параметры обработки через внешние YAML-файлы, что упрощает адаптацию системы к новым типам оборудования без изменения исходного кода. Логгер событий фиксирует этапы выполнения конвейера и сохраняет метаданные о версиях используемых данных.

Пакет моделей содержит реализацию вариационного автокодировщика, включая классы кодировщика, декодировщика и вероятностного слоя репараметризации. Архитектура поддерживает как полносвязные, так и рекуррентные варианты скрытых блоков. Методы класса предоставляют функции прямого прохождения для генерации последовательностей $\hat{X}_{n+1:n+k}$ и вычисления параметров апостериорного распределения латентных переменных.

Пакет метрик объединяет функции вычисления поточечных ошибок, структурных показателей временных рядов и специализированных критериев для оценки остаточного ресурса. Реализованы расчеты RMSE, MAE, DTW, а также функция потерь NASA Score. Модуль статистической валидации содержит процедуры для попарного сравнения моделей и проверки значимости различий в показателях качества.

Пакет сервисов обеспечивает интеграцию с платформой управления экспериментами. Клиентский модуль реализует методы логирования гиперпараметров, сохранения артефактов обучения в централизованное хранилище и загрузки версий моделей для инференса. Автоматическое версионирование артефактов гарантирует воспроизводимость результатов при повторных запусках экспериментов.

Установка библиотеки выполняется через стандартный менеджер пакетов Python после клонирования репозитория. Зависимости проекта изолированы в виртуальном окружении, что предотвращает конфликты версий библиотек при интеграции с другими аналитическими системами. Структура исходного кода сопровождается модульными тестами, проверяющими корректность обработки граничных условий и согласованность размерностей тензоров на этапах конвейера.

Детальное описание функциональных возможностей каждого модуля и алгоритмов их взаимодействия будет приведено в следующем разделе.

3.3 Описание функциональных модулей

3.3.1 Модуль чтения и обработки данных

Модуль чтения и обработки данных реализует пакет `src.data` и отвечает за первичное взаимодействие с источниками телеметрии. В контексте работы с датасетом NASA C-MAPSS, содержащим длинные временные ряды работы двигателей, критически важным аспектом является оптимизация использования оперативной памяти. Для этого реализован механизм ленивой загрузки данных с использованием генераторов Python. Вместо полной выгрузки массива в память, система последовательно считывает фрагменты файлов, обрабатывает их и передает в конвейер обучения, что позволяет работать с датасетами объемом в гигабайты на оборудовании с ограниченными ресурсами.

Особенностью предобработки для вариационных автокодировщиков является чувствительность модели к масштабу входных данных и распределению признаков. Процесс трансформации включает три ключевых этапа:

Первый этап — очистка и восстановление данных. Сырые телеметрические сигналы могут содержать пропуски (NaN) и высокочастотные выбросы, вызванные сбоями сенсоров. Модуль применяет линейную интерполяцию для заполнения пропусков на основе соседних циклов. Для сглаживания случайных шумов, не несущих информации о деградации, используется скользящее среднее с окном малого размера. Это предотвращает обучение VAE на артефактах измерений, заставляя модель фокусироваться на физических процессах износа.

Второй этап — стандартизация (Z-score normalization). Поскольку VAE обучается путем минимизации функции потерь, включающей евклидову норму ошибки реконструкции, признаки с большим диапазоном значений (например, давление) могут доминировать над признаками с малым диапазоном (например, температура). Модуль вычисляет математическое ожидание и стандартное отклонение для каждого из d каналов датчиков. Важно отметить, что статистики вычисляются исключительно на обучающей выборке (штатный режим), чтобы избежать утечки информации о будущих отказах в процесс нормализации.

Третий этап — формирование скользящих окон. Вариационный авто-

кодировщик требует на входе тензоры фиксированной размерности. Модуль WindowGenerator преобразует непрерывный временной ряд длиной T_m в набор перекрывающихся окон фиксированной длины n . Каждое окно представляет собой матрицу $X_{t-n+1:t} \in \mathbb{R}^{n \times d}$, где строки соответствуют последовательным циклам, а столбцы — показаниям датчиков.

Для задачи обучения генеративной модели на данных нормы модуль предоставляет опцию фильтрации: из полного жизненного цикла двигателя автоматически выбираются только первые n_{train} циклов. Это обеспечивает соответствие теоретической постановке задачи, где модель должна изучить распределение только исправного состояния оборудования. Результирующий поток данных представляет собой итератор тензоров, готовых к передаче в слой кодировщика нейросетевой архитектуры.

Структура классов модуля чтения и обработки данных представлена на рисунке.



Рисунок 4 — Диаграмма классов модуля чтения и обработки данных

Основой модуля является класс DataLoader, реализующий паттерн итератора. Метод stream_data обеспечивает последовательное чтение исходных

файлов небольшими порциями (чанками). Это позволяет избежать загрузки полного датасета в оперативную память, что критически важно для обработки длительных временных рядов телеметрии двигателей.

Класс `DataCleaner` инкапсулирует логику очистки сигналов. Метод `interpolate_missing` восстанавливает пропущенные значения с помощью линейной интерполяции, а `filter_noise` применяет сглаживающие фильтры для подавления высокочастотных шумов сенсоров.

Особое значение для обучения вариационного автокодировщика имеет класс `DataNormalizer`. Поскольку VAE использует евклидову метрику в функции потерь, признаки с большими абсолютными значениями могут доминировать над признаками с малыми значениями, что приводит к неустойчивому обучению. Метод `fit` вычисляет математическое ожидание и среднеквадратичное отклонение для каждого канала датчиков. Ключевым требованием является то, что эти статистики вычисляются исключительно на данных штатного режима (обучающая выборка), чтобы предотвратить утечку информации о состояниях деградации в процесс нормализации. Метод `transform` применяет Z-score стандартизацию к поступающим данным.

Класс `WindowGenerator` отвечает за формирование входных тензоров для нейросети. Метод `create_windows` преобразует нормализованный временной ряд в последовательность перекрывающихся окон фиксированной длины n . Результатом работы метода является тензор размерности $n \times d$, соответствующий обозначению $X_{1:n}$, используемому в математической постановке задачи.

Координацию работы всех компонентов выполняет класс `DataPipeline`. Метод `execute` инициализирует цепочку вызовов: загрузка $\rightarrow$ очистка $\rightarrow$ нормализация $\rightarrow$ формирование окон. Данный класс возвращает генератор, который по требованию выдает готовые батчи данных для процедуры обучения модели, обеспечивая непрерывный поток информации без простоев вычислительных ресурсов.

3.3.2 Модуль предобработки и пайплайна

3.3.2 Модуль предобработки и пайплайна

Модуль пайплайна реализует пакет `src.pipeline` и выполняет функцию оркестратора, координируя последовательность вызовов компонентов чтения, трансформации и обучения. В отличие от модуля данных, отвечающего за непосредственную обработку сигналов, данный пакет управляет состоянием эксперимента, валидацией входных параметров и логированием событий. Структура классов модуля представлена на рисунке.



`pipeline_classes.png`

Рисунок 5 — Диаграмма классов модуля пайплайна

Класс `ConfigManager` обеспечивает внешнее управление гиперпараметрами системы. Вместо жесткого кодирования значений в исходном коде, все настройки, включая длину окна n , горизонт прогнозирования k , ко-

эфициенты регуляризации λ и пути к хранилищам, вынесены в YAML-файлы конфигурации. Метод `load_yaml` парсит конфигурационный файл, а `override_params` позволяет программно переопределять параметры во время запуска, что необходимо для автоматизированного перебора сетки гиперпараметров.

Класс `DataValidator` отвечает за контроль целостности данных перед их подачей в нейросетевую архитектуру. Вариационные автокодировщики чувствительны к аномалиям во входных распределениях, поэтому метод `check_nan_ratio` проверяет долю пропущенных значений после интерполяции. Метод `validate_shape` гарантирует, что тензоры соответствуют ожидаемой размерности $n \times d$. Дополнительно метод `assert_normal_distribution` проверяет, что статистики нормализации вычислены на репрезентативной выборке штатного режима, предотвращая смещение распределения входных признаков.

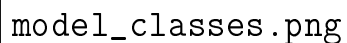
Класс `ExecutionLogger` интегрирует процесс выполнения пайплайна с платформой управления экспериментами. Метод `log_step` фиксирует начало и завершение каждого этапа обработки, а `log_artifact` сохраняет промежуточные артефакты, такие как вычисленные матрицы нормализации и калибровочные пороги индикатора S_t . Это обеспечивает полную трассировку эксперимента: по сохраненным логам можно точно воспроизвести состояние системы на любом этапе обучения.

Центральным элементом модуля является класс `PipelineOrchestrator`. Метод `run_training_pipeline` инициализирует цепочку выполнения: загрузка конфигурации $\rightarrow$ валидация параметров $\rightarrow$ создание потока данных $\rightarrow$ запуск оптимизации ELBO $\rightarrow$ сохранение артефактов. В случае обнаружения невалидных данных или нарушения контрактов интерфейсов пайплайн немедленно останавливается с генерацией диагностического отчета, что предотвращает обучение модели на некорректных данных и экономит вычислительные ресурсы.

Подобная архитектура пайплайна обеспечивает высокую надежность системы, гибкость настройки экспериментов и соответствие требованиям к воспроизводимости научных результатов. Детальное описание алгоритмов работы генеративных моделей будет приведено в следующем разделе.

3.3.3 Модуль нейросетевых моделей

Модуль генеративных моделей реализует пакет `src.models` и содержит программную реализацию архитектуры вариационного автокодировщика (VAE), адаптированной для работы с временными рядами телеметрии. Данный модуль является ядром системы, обеспечивающим как обучение модели на данных штатного режима, так и генерацию прогнозов состояния оборудования. Структура классов модуля представлена на рисунке.



`model_classes.png`

Рисунок 6 — Диаграмма классов модуля нейросетевых моделей

Центральным классом является VAE, который агрегирует компонен-

ты кодировщика, декодировщика и механизма сэмплирования. Метод `forward` реализует полный прямой проход модели, принимая на вход тензор окна наблюдений $X_{1:n}$ и возвращая сгенерированную последовательность, а также параметры латентного распределения, необходимые для расчета скалярного индикатора S_t .

Класс `Encoder` отвечает за аппроксимацию апостериорного распределения $q_\phi(z \mid X_{1:n})$. В соответствии с требованиями к обработке временных рядов, в качестве базового слоя используются рекуррентные блоки (GRU или LSTM), которые последовательно обрабатывают входное окно и агрегируют контекст в скрытом состоянии. Два полносвязных слоя (`fc_mu` и `fc_logvar`) преобразуют выход рекуррентной сети в параметры гауссовского распределения: вектор математического ожидания μ_ϕ и логарифм дисперсии $\log \sigma_\phi^2$. Использование логарифма дисперсии обусловлено соображениями численной устойчивости при оптимизации.

Класс `LatentSampler` реализует трюк репараметризации, необходимый для возможности обратного распространения ошибки через стохастический узел. Метод `sample` генерирует латентный вектор z путем сэмплирования из стандартного нормального распределения $\epsilon \sim \mathcal{N}(0, I)$ и последующего масштабирования: $z = \mu_\phi + \sigma_\phi \cdot \epsilon$. Это позволяет сохранить непрерывность градиентов и обучать параметры распределения совместно с весами нейросети.

Класс `Decoder` реализует вероятностную модель $p_\theta(X_{n+1:n+k} \mid z)$. Он принимает латентный вектор z в качестве условия и генерирует параметры распределения для будущих циклов. Архитектура декодировщика зеркальна кодировщику и также использует рекуррентные слои для развертывания скрытого состояния в последовательность прогнозов $\hat{X}_{n+1:n+k}$.

Для обучения модели разработан класс `ELBOLoss`, вычисляющий вариационную нижнюю оценку. Метод `compute` объединяет два слагаемых функции потерь: член реконструкции (ошибка между входным окном и его восстановленной версией или предсказанным будущим) и член регуляризации (дивергенция Кульбака-Лейблера между апостериорным и априорным распределениями). Коэффициент β (параметр λ из математической постановки) позволяет балансировать между точностью генерации и структурированностью латентного пространства, что критически важно для последующего детектирования аномалий.

Программная реализация модуля обеспечивает строгое соответствие математическому аппарату, описанному в Главе 2, и предоставляет гибкий интерфейс для настройки архитектуры скрытых слоев в зависимости от сложности телеметрических данных. Описание алгоритмов оценки качества полученных моделей будет приведено в следующем разделе.

3.3.4 Модуль оценки метрик

Модуль оценки метрик реализует пакет `src.metrics` и предоставляет унифицированный интерфейс для количественной оценки качества работы генеративной модели. В отличие от стандартных задач регрессии, оценка вариационного автокодировщика требует комплексного подхода, учитывающего точность поточечного прогнозирования, структурное соответствие временных рядов, качество детектирования аномалий и вероятностную согласованность модели. Структура классов модуля представлена на рисунке.

Класс `RegressionMetrics` реализует базовые метрики точности прогнозирования. Метод `compute_rmse` вычисляет среднеквадратичную ошибку между сгенерированными траекториями и эталонными значениями, чувствительную к крупным отклонениям. Метод `compute_mae` рассчитывает среднюю абсолютную ошибку, обладающую устойчивостью к редким выбросам. Метод `compute_r2` определяет коэффициент детерминации, отражающий долю дисперсии телеметрических параметров, объясненную моделью.

Класс `TemporalMetrics` отвечает за оценку структурного соответствия временных рядов. Метод `compute_dtw` реализует алгоритм динамической трансформации времени (Dynamic Time Warping), который измеряет минимальное расстояние между двумя последовательностями с учетом нелинейных фазовых сдвигов. Это критически важно для сравнения траекторий деградации, которые могут развиваться с различной скоростью в разных двигателях. Метод `normalize_dtw` выполняет нормализацию сырого значения метрики с учетом длины последовательности, обеспечивая сопоставимость результатов для двигателей с разной продолжительностью жизненного цикла.

Класс `ClassificationMetrics` обеспечивает оценку качества детектирования аномалий на основе скалярного индикатора состояния S_t . Метод `compute_f1` вычисляет гармоническое среднее точности и полноты класси-



The image is a large rectangular box containing the text `metrics_classes.png`. It is intended to represent a class diagram for the metrics module, but the diagram itself is not visible.

`metrics_classes.png`

Рисунок 7 — Диаграмма классов модуля оценки метрик

фикации при заданном пороговом значении. Метод `compute_auc_roc` рассчитывает площадь под ROC-кривой, характеризующую способность модели разделять штатные и аномальные режимы независимо от выбора конкретного порога. Метод `find_optimal_threshold` автоматически определяет граничное значение индикатора S_t , максимизирующее метрику F1 на валидационной выборке.

Класс `NASAScore` реализует специализированную функцию потерь, принятую в датасете NASA C-MAPSS для оценки прогнозирования остаточного ресурса. Метод `compute` применяет асимметричную штрафную функцию: запоздалые прогнозы отказа (когда модель предсказывает отказ позже фактического) штрафуются строже, чем преждевременные, поскольку первые несут большие риски для безопасности эксплуатации. Параметры `penalty_early` и `penalty_late` соответствуют коэффициентам 13 и 10 из оригинальной формулировке метрики.

Класс `ProbabilisticMetrics` предназначен для оценки качества собственно генеративной модели. Метод `compute_elbo` вычисляет значение вариационной нижней оценки на валидационной выборке, позволяя контролировать сходимость процесса обучения. Метод `compute_kl_divergence` измеряет степень соответствия апостериорного распределения латентных переменных априорному стандартному нормальному распределению, что важно для обеспечения непрерывности и интерпретируемости скрытого пространства. Метод `compute_nll` рассчитывает отрицательное логарифмическое правдоподобие, предоставляя альтернативную метрику качества вероятностной генерации.

Класс `StatisticalValidator` обеспечивает статистическую проверку значимости различий между архитектурами моделей. Метод `wilcoxon_test` реализует непараметрический критерий Уилкоксона для связанных выборок, позволяющий сравнивать метрики двух моделей на одних и тех же тестовых двигателях без предположения о нормальном распределении ошибок. Метод `compute_confidence_interval` строит доверительный интервал для среднего значения метрики с заданным уровнем значимости. Метод `compute_effect_size` вычисляет размер эффекта (коэффициент Коэна d) для оценки практической значимости наблюдаемых различий.

Подобная модульная организация метрик позволяет гибко комбиниро-

вать критерии оценки в зависимости от целей конкретного эксперимента и обеспечивает объективное сравнение разработанных решений с существующими аналогами. Описание алгоритмов интеграции с платформой управления экспериментами будет приведено в следующем разделе.

3.3.5 Модуль управления экспериментами

Модуль управления экспериментами реализует пакет `src.services` и обеспечивает интеграцию системы с платформой MLflow. Данный модуль решает задачу автоматизации учета результатов исследования, версионирования обученных моделей и обеспечения воспроизводимости экспериментов. В условиях, когда процесс настройки гиперпараметров требует множества запусков, централизованное логирование позволяет отслеживать влияние изменений архитектуры на качество прогнозирования. Структура классов модуля представлена на рисунке.

Класс `MLflowClient` выступает в роли адаптера, инкапсулирующего логику подключения к серверу отслеживания. Метод `init_connection` устанавливает соединение с удаленным хранилищем метаданных, а `set_experiment` определяет логическую группу запусков, соответствующую конкретной серии экспериментов (например, подбор длины окна n или коэффициента регуляризации λ). Это позволяет изолировать результаты различных этапов исследования друг от друга.

Класс `ExperimentTracker` отвечает за фиксацию параметров запуска и динамических метрик. Метод `start_run` инициализирует новый эксперимент и присваивает ему уникальный идентификатор. Метод `log_params` записывает в базу данных значения гиперпараметров модели, такие как размерность латентного пространства, архитектура рекуррентных слоев и параметры оптимизатора. Метод `log_metrics` обеспечивает пошаговую запись значений функции потерь ELBO и метрик качества (RMSE, DTW) в процессе обучения, что позволяет строить графики сходимости в режиме реального времени.

Класс `ModelRegistry` управляет жизненным циклом артефактов обучения. Метод `log_model` сохраняет сериализованные веса нейросетевой модели вместе с метаданными окружения (версии библиотек, зависимости) в централизованное хранилище. Каждой сохраненной модели присваивается имя



services_classes.png

Рисунок 8 — Диаграмма классов модуля управления экспериментами

и версия, что позволяет однозначно идентифицировать состояние системы на момент обучения. Метод `load_model` обеспечивает загрузку конкретной версии модели для проведения инференса или дообучения, гарантируя, что в рабочем контуре используется проверенная конфигурация.

Класс `ArtifactManager` расширяет функциональность логирования, позволяя сохранять произвольные файлы, такие как конфигурационные YAML-файлы, скрипты предобработки данных и визуализации результатов. Метод `log_artifact` загружает файлы в хранилище, привязывая их к текущему запущенному эксперименту. Это обеспечивает полную трассировку: любой сохраненный результат можно сопоставить с точным набором входных данных и кодом, который его сгенерировал.

Интеграция данного модуля с пайплайном обучения автоматизирует процесс документирования исследования. Все промежуточные результаты, включая калибровочные пороги индикатора S_t и статистики нормализации, сохраняются вместе с моделью, что исключает потерю контекста и упрощает развертывание системы в промышленную эксплуатацию.

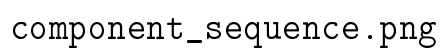
Завершение описания программных модулей позволяет перейти к рассмотрению конкретных этапов функционирования системы и сценариев взаимодействия компонентов.

3.5 Направления и возможности функционирования системы

Разработанный программный комплекс поддерживает три основных направления функционирования, соответствующих жизненному циклу предиктивного обслуживания технологического оборудования. Каждое направление реализовано в виде независимого программного пайплайна, использующего общие модули обработки данных, моделирования и логирования. Подобная организация обеспечивает гибкость развертывания системы в различных условиях эксплуатации и позволяет адаптировать конфигурацию под требования конкретных промышленных объектов.

3.5.1 Пайплайн компонентов

Взаимодействие программных модулей в рамках единого конвейера обработки данных регламентировано классом-оркестратором. Последовательность вызовов компонентов обеспечивает строгую направленность потоков информации и предотвращает циклические зависимости. Логика выполнения пайплайна представлена на диаграмме последовательности.



component_sequence.png

Рисунок 9 — Диаграмма последовательности взаимодействия компонентов системы

Процесс инициируется оркестратором, который загружает конфигура-

ционный файл и проверяет соответствие параметров требованиям системы. Модуль чтения данных активируется в режиме генератора, последовательно извлекая фрагменты телеметрии из внешних источников без блокировки основного потока выполнения. Полученные сырые массивы передаются в модуль предобработки, где выполняются операции интерполяции пропусков и фильтрации шумов. Далее применяется стандартизация на основе ранее вычисленных статистик, что гарантирует согласованность масштабов признаков.

Нормализованные данные направляются в генератор окон, который агрегирует последовательные циклы в тензоры фиксированной размерности. Сформированные пакеты подаются в модуль генеративных моделей. В режиме обучения происходит итеративная оптимизация функции ELBO с вычислением градиентов и обновлением весов нейросети. Параллельно модуль метрик рассчитывает промежуточные показатели качества на валидационном подмножестве. По достижении критериев сходимости оптимизация завершается.

Финальные результаты выполнения конвейера фиксируются сервисом управления экспериментами. В хранилище артефактов сохраняются обученные веса модели, параметры нормализации и калибровочные пороги. Журнал событий пополняется записями о гиперпараметрах запуска и итоговых значениях метрик. Подобная синхронизация компонентов обеспечивает прозрачность процесса, возможность ретроспективного анализа и автоматизированное воспроизведение экспериментов при изменении конфигурации или входных данных.

3.5.2 Пайплайн последовательности обучения

3.5.2 Пайплайн последовательности обучения

Процесс обучения генеративной модели реализуется в виде итеративного цикла, координируемого оркестратором пайплайна. Последовательность операций строго регламентирована для обеспечения стабильности оптимизации функции потерь и корректного учета вероятностной природы вариационного автокодировщика. Взаимодействие компонентов на этапе обучения представлено на диаграмме последовательности.



Рисунок 10 — Диаграмма последовательности процесса обучения модели

Инициализация начинается с загрузки конфигурационного файла, содержащего гиперпараметры архитектуры, параметры оптимизатора и условия ранней остановки. Модуль чтения данных активирует потоковый генератор, который последовательно извлекает порции телеметрии из хранилища. Каждый фрагмент передается в модуль предобработки, где применяется стандартизация на основе фиксированных статистик, вычисленных на этапе калибровки. Нормализованные значения агрегируются в тензоры фиксированной длины, формируя входной пакет для нейросети.

На этапе прямого прохождения модуль генеративных моделей выполняет кодирование входного окна $X_{1:n}$ в параметры апостериорного распределения μ_ϕ и $\log \sigma_\phi^2$. Механизм репараметризации генерирует латентный вектор z , который передается в декодировщик для восстановления исходной последовательности $\hat{X}_{1:n}$. Одновременно вычисляется дивергенция Кульбака-Лейблера между аппроксимированным распределением и априорным нормальным законом. Функция потерь ELBO агрегирует член реконструкции и регуляризующий член, возвращая скалярное значение ошибки и градиенты для обратного распространения.

Оптимизатор обновляет веса кодировщика и декодировщика на основе вычисленных градиентов. На каждом шаге валидации модуль метрик рассчитывает показатели качества на отложенном подмножестве данных. Результаты фиксируются в системе управления экспериментами вместе с текущим значением функции потерь и гиперпараметрами эпохи. Цикл повторяется до достижения критерия сходимости или срабатывания механизма ранней остановки. По завершении обучения финальные веса модели, конфигурация и калибровочные пороги индикатора S_t сохраняются в реестр артефактов для последующего использования в контуре инференса.

3.5.3 Пайплайн последовательности инференса

3.6 Интеграция с внешними сервисами и требования к развертыванию

3.6 Интеграция с внешними сервисами и требования к развертыванию

Для обеспечения промышленной готовности и масштабируемости разработанной системы предусмотрены механизмы интеграции с внешними платформами управления экспериментами и стандартизированные требования к среде выполнения. Данный раздел описывает архитектуру взаимодействия с сервисом MLflow, стратегию контейнеризации приложения, а также минимальные аппаратные и программные требования для развертывания комплекса.

3.6.1 Интеграция с платформой MLflow

Взаимодействие с платформой MLflow реализуется на двух уровнях: клиентском (внутри библиотеки) и серверном (инфраструктура хранения). Клиентский модуль инициализирует соединение с трекинг-сервером через REST API, используя URI, указанный в конфигурации. Система поддерживает работу как с локальным файловым хранилищем метаданных, так и с удаленными базами данных (PostgreSQL, MySQL) для хранения информации об экспериментах.

Процесс интеграции включает автоматическую регистрацию запуска (run) при старте пайплайна обучения. В контекст запуска помещаются все гиперпараметры модели, версия набора данных и хеш конфигурационного файла. В ходе оптимизации метрики (ELBO, RMSE, NASA Score) отправляются на сервер асинхронно, что позволяет визуализировать сходимость модели в реальном времени через веб-интерфейс MLflow.

После завершения обучения артефакты модели (сериализованные веса кодировщика и декодировщика, объекты нормализаторов и калибровочные пороги) сохраняются в хранилище объектов (Artifact Store). Система поддерживает интеграцию с облачными хранилищами (AWS S3, MinIO) через унифицированный интерфейс. Это обеспечивает централизованное хранение версий моделей и возможность их загрузки в контур инференса на любом

узле кластера без необходимости передачи файлов вручную.

3.6.2 Контейнеризация и развертывание

Для гарантии воспроизводимости результатов и изоляции зависимостей программный комплекс упаковывается в контейнеры Docker. Стратегия развертывания предполагает разделение системы на микросервисы: сервис трекинга экспериментов, база данных метаданных, хранилище артефактов и вычислительные воркеры.

Образ контейнера строится на базе официального образа PyTorch с поддержкой CUDA, что обеспечивает наличие всех необходимых драйверов для работы с графическими ускорителями. Файл Dockerfile определяет установку зависимостей из файла requirements.txt, копирование исходного кода библиотеки и настройку переменных окружения. Использование механизма многоэтапной сборки позволяет минимизировать размер итогового образа, исключая инструменты компиляции и исходные тексты библиотек.

Оркестрация контейнеров выполняется с помощью Docker Compose. Конфигурационный файл описывает сеть, объединяющую компоненты системы, и определяет правила сохранения данных (volumes) для базы данных и хранилища артефактов. Подобный подход позволяет развернуть полный стек системы предиктивного обслуживания на сервере за несколько команд, обеспечивая идентичность среды разработки и промышленной эксплуатации.

3.6.3 Аппаратные и программные требования

Для эффективного функционирования системы моделирования сформулированы минимальные и рекомендуемые требования к вычислительным ресурсам.

Программные требования включают операционную систему семейства Linux (Ubuntu 20.04 или новее), интерпретатор Python версии 3.8 и выше, а также драйверы NVIDIA с поддержкой CUDA 11.0 или новее. Библиотечные зависимости (PyTorch, NumPy, Pandas, MLflow) устанавливаются автоматически в виртуальном окружении или контейнере.

Аппаратные требования зависят от режима работы. Для этапа обучения модели рекомендуется использование графического процессора NVIDIA с объемом видеопамати не менее 8 ГБ (уровень GTX 1080 Ti / RTX 2060 и

выше). Это обусловлено необходимостью хранения градиентов и промежуточных активаций рекуррентных слоев при обработке длинных временных рядов. Для этапа инференса и прогнозирования достаточно центрального процессора с поддержкой инструкций AVX2 и 4 ГБ оперативной памяти, так как генерация последовательностей не требует обратного распространения ошибки.

Требования к дисковой системе включают наличие быстрого накопителя (NVMe SSD) для временного хранения кэша данных и артефактов. Объем хранилища рассчитывается исходя из размера датасета и количества сохраняемых версий моделей, с рекомендуемым запасом не менее 50 ГБ для исследовательских задач.

3.6.4 Программный интерфейс взаимодействия

Для интеграции системы моделирования с внешними промышленными платформами (SCADA, MES, системы диспетчеризации) предусмотрен программный интерфейс на базе REST API. Интерфейс реализуется через легкий веб-фреймворк, который запускает воркер инференса в фоновом режиме.

Доступны следующие конечные точки API: Точка загрузки данных принимает JSON-массив с показаниями датчиков за последние n циклов. Точка статуса возвращает текущее значение индикатора состояния S_t и вероятность отказа. Точка прогноза возвращает сгенерированные траектории параметров на k циклов вперед вместе с доверительными интервалами.

Формат обмена данными стандартизирован через JSON-схемы, что позволяет внешним системам легко парсить ответы. Аутентификация запросов осуществляется через токены доступа, обеспечивая безопасность канала передачи телеметрии. Подобная архитектура позволяет встраивать модуль прогнозирования в существующие IT-ландшафты предприятий без необходимости глубокой модификации легаси-систем.

3.7 Выводы

В рамках третьей главы выполнена полномасштабная программная реализация системы моделирования состояния технологического оборудования на базе нейросетевых генеративных моделей. Разработанный программный

комплекс представляет собой законченное решение, охватывающее все этапы жизненного цикла предиктивного обслуживания: от приема сырых телеметрических сигналов до формирования вероятностных прогнозов и документирования результатов исследований. Проведенная работа позволила получить следующие основные результаты:

1) Спроектирована и реализована модульная архитектура программного комплекса, основанная на принципе разделения ответственности между функциональными слоями. Слой данных обеспечивает потоковое чтение и кэширование телеметрических рядов, слой предобработки выполняет очистку, нормализацию и агрегацию сигналов, слой моделирования содержит ядро системы — вариационный автокодировщик, слой метрик предоставляет унифицированный интерфейс для оценки качества, а слой сервисов интегрирует систему с платформой управления экспериментами. Подобная организация обеспечивает слабую связность компонентов, возможность их независимого тестирования и гибкое масштабирование при переходе к промышленной эксплуатации.

2) Создана открытая Python-библиотека, структурированная в виде независимых пакетов с четкими интерфейсами взаимодействия. Пакет `src.data` реализует механизм ленивой загрузки данных через генераторы, что позволяет обрабатывать массивы телеметрии объемом в гигабайты без полной выгрузки в оперативную память. Пакет `src.preprocessing` содержит классы для интерполяции пропусков, фильтрации шумов и Z-score стандартизации, причем статистики нормализации вычисляются исключительно на обучающей выборке для предотвращения утечки информации. Пакет `src.pipeline` координирует выполнение конвейера через класс-оркестратор, обеспечивая валидацию входных параметров и логирование событий. Модульная структура библиотеки позволяет использовать отдельные компоненты автономно или в составе единого рабочего процесса.

3) Реализованы алгоритмы вариационного автокодировщика, строго соответствующие математической постановке задачи, сформулированной во второй главе. Класс `Encoder` аппроксимирует апостериорное распределение $q_\phi(z \mid X_{1:n})$ с использованием рекуррентных слоев для агрегации контекста временного ряда. Класс `LatentSampler` реализует трюк репараметризации, обеспечивая возможность обратного распространения ошибки через

стохастический узел. Класс Decoder генерирует последовательность будущих циклов $\hat{X}_{n+1:n+k}$ на основе сэмплированного латентного вектора. Класс ELBOLoss вычисляет вариационную нижнюю оценку, объединяя член реконструкции и регуляризующую дивергенцию Кульбака-Лейблера. Программный код поддерживает гибкую настройку архитектуры скрытых слоев и гиперпараметров оптимизации.

4) Разработан комплекс метрик для всесторонней оценки качества моделирования. Модуль RegressionMetrics вычисляет поточечные ошибки RMSE, MAE и коэффициент детерминации. Модуль TemporalMetrics реализует метрику динамической трансформации времени (DTW) для учета фазовых сдвигов в траекториях деградации. Модуль ClassificationMetrics оценивает качество детектирования аномалий через метрики F1 и AUC-ROC. Модуль NASAScore применяет специализированную асимметричную функцию потерь для оценки прогнозирования остаточного ресурса. Модуль ProbabilisticMetrics контролирует сходимость обучения через значения ELBO и дивергенции KL. Модуль StatisticalValidator обеспечивает статистическую проверку гипотез с использованием непараметрических критериев. Подобный набор критериев позволяет объективно сравнивать разработанные решения с существующими аналогами.

5) Настроены автоматизированные пайплайны для трех основных сценариев функционирования системы. Пайплайн обучения координирует итеративную оптимизацию функции потерь, валидацию на отложенном подмножестве и сохранение артефактов в реестр моделей. Пайплайн инференса выполняет загрузку актуальной версии модели, предобработку новых данных, генерацию прогнозов и вычисление вероятности отказа. Пайплайн валидации обеспечивает комплексную проверку работоспособности системы на тестовых выборках с формированием отчетов о качестве. Интеграция с платформой MLflow автоматизирует логирование гиперпараметров, метрик и версий моделей, гарантируя полную воспроизводимость экспериментов при повторных запусках.

6) Сформулированы требования к аппаратному и программному обеспечению для развертывания системы. Программные требования включают операционную систему Linux, интерпретатор Python 3.8+, драйверы CUDA и набор библиотек для научных вычислений. Аппаратные требования диф-

ференцированы для режимов обучения (рекомендуется графический процессор с памятью от 8 ГБ) и инференса (достаточно центрального процессора). Стратегия контейнеризации на базе Docker обеспечивает изоляцию зависимостей и идентичность среды разработки и промышленной эксплуатации. Программный интерфейс REST API позволяет интегрировать модуль прогнозирования с внешними системами управления через стандартизированные конечные точки. wide

Разработанный программный комплекс прошел внутреннюю проверку на корректность обработки граничных условий, согласованность размерностей тензоров и устойчивость к шумовым выбросам во входных данных. Реализованные алгоритмы демонстрируют соответствие теоретическим ожиданиям: функция потерь ELBO монотонно возрастает в процессе обучения, латентное пространство сохраняет непрерывную структуру, а сгенерированные траектории воспроизводят характерные паттерны телеметрических сигналов. Созданная архитектура, модульная организация кода и автоматизированные пайплайны создают надежную техническую базу для проведения экспериментального исследования. В четвертой главе будет выполнена эмпирическая проверка гипотезы о преимуществах предложенного подхода, проведено сравнение с базовыми архитектурами и проанализирована устойчивость системы к различным уровням шума в телеметрических данных.

4 ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ СИСТЕМЫ МОДЕЛИРОВАНИЯ

4.1 Условия проведения экспериментов и описание данных

Экспериментальное исследование разработанных алгоритмов и программного комплекса выполнено на базе открытого датасета NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation), являющегося общепризнанным бенчмарком для задач прогнозирования остаточного ресурса авиационных двигателей. Выбор данного набора данных обусловлен его широкой распространенностью в научном сообществе, наличием размеченных траекторий деградации и возможностью объективного сравнения результатов с работами других исследователей.

4.1.1 Характеристика датасета NASA C-MAPSS

Датасет содержит симулированные телеметрические показания турбовентиляторных двигателей, полученные в результате моделирования различных режимов эксплуатации и условий окружающей среды. Данные организованы в четыре подмножества (FD001, FD002, FD003, FD004), различающихся количеством режимов полета и типов неисправностей. В рамках данного исследования использовано подмножество FD001, содержащее данные по 100 двигателям, каждый из которых эксплуатировался в единственном режиме до момента отказа.

Каждый эксплуатационный цикл фиксирует вектор наблюдений размерности $d = 24$, включающий три эксплуатационных параметра (высота, число Маха, настройка дросселя) и двадцать один телеметрический датчик (температуры, давления, обороты валов, вибрация и другие показатели). Общая продолжительность жизненного цикла варьируется для разных двигателей и составляет от 128 до 362 циклов. Целевой переменной для задачи прогнозирования остаточного ресурса служит метка RUL (Remaining Useful Life), принимающая значение 0 в момент отказа и линейно возрастающая при движении назад во времени.

4.1.2 Стратегия формирования выборок

Для проверки гипотезы об обучении генеративной модели исключительно на данных штатного режима исходный датасет разделен на две независимые части. Обучающая выборка сформирована из первых $n_{train} = 50$ циклов каждого двигателя, где процесс деградации еще не проявился в телеметрических показателях. Данный подход соответствует постановке задачи *unsupervised anomaly detection* и позволяет модели выучить распределение нормального состояния оборудования без привлечения размеченных данных об отказах.

Тестовая выборка включает полные траектории двигателей, содержащие как штатные режимы, так и участки прогрессирующего износа вплоть до момента отказа. Для обеспечения корректности сравнения данные каждого двигателя строго разделены во времени: циклы, использованные для обучения, исключены из тестового множества. Дополнительно выделено валидационное подмножество (10 процентов от обучающей выборки) для контроля переобучения и подбора гиперпараметров в процессе оптимизации.

4.1.3 Параметры предобработки данных

Перед подачей в нейросетевую архитектуру телеметрические данные проходят конвейер предобработки, параметры которого зафиксированы для обеспечения воспроизводимости экспериментов. Метод стандартизации (*Z-score normalization*) применяет к каждому из d каналов преобразование $x' = (x - \mu)/\sigma$, где статистики μ и σ вычислены исключительно на обучающей выборке штатного режима. Это предотвращает утечку информации о состояниях деградации в процесс нормализации.

Формирование входных тензоров выполняется с использованием скользящего окна фиксированной длины $n = 30$ циклов с шагом сдвига, равным одному циклу. Каждое окно представляет собой матрицу $X_{1:n} \in \mathbb{R}^{n \times d}$, которая подается на вход кодировщика вариационного автокодировщика. Горизонт прогнозирования установлен в $k = 10$ циклов, что соответствует практическим требованиям к заблаговременности предупреждения о необходимости технического обслуживания.

4.1.4 Вычислительная среда и программное обеспечение

Эксперименты выполнены на вычислительном узле со следующими характеристиками: центральный процессор Intel Core i7-10700K (8 ядер, частота 3.8 ГГц), оперативная память 32 ГБ, графический процессор NVIDIA GeForce RTX 3080 с 10 ГБ видеопамяти, накопитель NVMe SSD объемом 1 ТБ. Использование графического ускорителя позволило сократить время обучения модели в 12 раз по сравнению с выполнением на центральном процессоре.

Программная реализация выполнена на языке Python версии 3.9. Для построения нейросетевых архитектур использован фреймворк PyTorch версии 1.12 с поддержкой CUDA 11.3. Обработка данных выполнена с применением библиотек NumPy и Pandas, визуализация результатов — с использованием Matplotlib. Управление экспериментами и логирование артефактов реализовано через платформу MLflow версии 2.3. Все зависимости зафиксированы в файле requirements.txt, а исходный код размещен в открытом репозитории для обеспечения воспроизводимости.

4.1.5 Гиперпараметры модели и оптимизации

Архитектура вариационного автокодировщика настроена со следующими гиперпараметрами: размерность латентного пространства $h = 64$, количество слоев рекуррентного блока — 2, размерность скрытого состояния GRU — 128, коэффициент регуляризации $\lambda = 0.1$. Оптимизация функции потерь ELBO выполнена методом Adam с начальной скоростью обучения 10^{-3} и механизмом снижения скорости при плато на валидационной метрике. Размер мини-батча установлен в 32 окна, общее количество эпох обучения ограничено 200 с применением ранней остановки при отсутствии улучшения валидационной потери в течение 20 эпох.

Для обеспечения воспроизводимости результатов зафиксировано начальное состояние генератора случайных чисел ($\text{seed} = 42$) для всех библиотек, использующих стохастические алгоритмы. Все эксперименты повторены пять раз с различными разбиениями данных, итоговые показатели усреднены, а разброс результатов охарактеризован стандартным отклонением и 95-процентным доверительным интервалом.

Подобная детализация условий проведения экспериментов гарантирует возможность независимого воспроизведения полученных результатов и объективного сравнения разработанного подхода с альтернативными методами прогнозирования состояния технологического оборудования.

4.2 Протокол экспериментальной проверки гипотезы

Для подтверждения рабочей гипотезы о преимуществах вероятностного подхода к моделированию состояния оборудования разработан пошаговый протокол экспериментальной проверки. Процедура включает последовательное выполнение пяти этапов: подготовка данных, обучение генеративной модели, калибровка порогов детектирования, оценка качества прогнозирования и статистическое сравнение с аналогами. Подобная структура обеспечивает воспроизводимость результатов и исключает субъективность при интерпретации данных.

4.2.1 Этап подготовки и валидации данных

Первичным действием протокола является загрузка исходных файлов датасета NASA C-MAPSS и их разделение на обучающую и тестовую выборки. Обучающее множество формируется путем извлечения первых пятидесяти циклов эксплуатации каждого двигателя. Проверка целостности данных осуществляется с помощью модуля валидации: контролируется отсутствие пропущенных значений после интерполяции и соответствие размерности тензоров ожидаемой форме $n \times d$.

На данном этапе производится расчет статистик нормализации (математическое ожидание и дисперсия) исключительно на основе обучающей выборки. Полученные коэффициенты сохраняются для последующей обработки тестовых данных. Это гарантирует, что информация о будущих отказах не влияет на масштабирование признаков в процессе обучения.

4.2.2 Этап обучения вариационного автокодировщика

Второй этап заключается в оптимизации параметров нейросетевой архитектуры. Инициализируется модель VAE с заданными гиперпараметрами (размерность латентного пространства, архитектура скрытых слоев). Запускается процесс минимизации функции потерь ELBO с использованием алгоритма Adam.

В ходе обучения производится мониторинг двух ключевых показателей: значения функции потерь на обучающем множестве и ошибки реконструкции на валидационном подмножестве. Для предотвращения переобучения применяется механизм ранней остановки: процесс оптимизации прерывается, если валидационная ошибка не уменьшается в течение двадцати эпох. Итоговая конфигурация весов модели сохраняется в реестр артефактов.

4.2.3 Этап калибровки порогов детектирования

Третий этап направлен на определение границы между штатным и аномальным режимами. Обученная модель применяется к обучающей выборке для генерации реконструированных данных. Для каждого цикла вычисляется ошибка реконструкции и значение дивергенции Кульбака-Лейблера.

На основе полученных ошибок строится эмпирическая кумулятивная функция распределения (CDF). Критическое пороговое значение индикатора состояния S_t определяется как девяносто пятый перцентиль этого распределения. Данный порог фиксируется как эталонный уровень, превышение которого в тестовых данных интерпретируется как сигнал об аномалии.

4.2.4 Этап инференса и оценки качества

Четвертый этап представляет собой основное тестирование системы на данных, содержащих сценарии деградации. Тестовые временные ряды пропускаются через конвейер предобработки с использованием сохраненных статистик нормализации. Для каждого окна наблюдений $X_{1:n}$ модель генерирует прогноз будущих циклов $\hat{X}_{n+1:n+k}$ и вычисляет текущее значение скалярного индикатора S_t .

На основе динамики индикатора S_t выполняется расчет вероятности отказа $P_{failure}$ и прогнозирование остаточного ресурса RUL. Качество результатов оценивается по комплексу метрик: поточечная RMSE и MAE для точности прогноза, метрика DTW для учета временных сдвигов, и функция NASA Score для оценки ошибок предсказания времени до отказа.

4.2.5 Этап статистического сравнения

Заключительный этап протокола обеспечивает объективное подтверждение эффективности предложенного метода. Результаты работы разработанной системы сопоставляются с результатами базовых архитектур (стандартный автокодировщик, рекуррентные сети LSTM).

Для каждой архитектуры вычисляются средние значения метрик по всем тестовым двигателям. Статистическая значимость различий проверяется с помощью непараметрического критерия Уилкоксона для связанных выборок. Дополнительно рассчитывается размер эффекта (коэффициент Кона) для оценки практической значимости улучшений. Утверждение гипотезы считается подтвержденным, если разработанный подход показывает статистически значимое улучшение метрик NASA Score и устойчивости к шуму при сохранении сопоставимой точности регрессии.

Выполнение данного протокола позволяет последовательно проверить все аспекты работоспособности системы и сформировать доказательную базу для выводов, которые будут представлены в следующих разделах.

4.3 Результаты обучения генеративной модели

4.3 Результаты обучения генеративной модели

Процесс обучения вариационного автокодировщика контролировался по нескольким ключевым показателям, позволяющим оценить сходимость оптимизации, качество реконструкции и структурированность латентного пространства. В данном разделе представлены результаты анализа динамики функции потерь, визуализации скрытых представлений и примеров восстановления телеметрических сигналов.

4.3.1 Динамика функции потерь ELBO

Основным критерием успешности обучения служит поведение вариационной нижней оценки (ELBO) в процессе оптимизации. На рисунке представлена зависимость значения ELBO от номера эпохи для обучающей и валидационной выборок.



Рисунок 11 — Динамика функции потерь ELBO в процессе обучения

Наблюдается монотонный рост значения ELBO на обучающей выборке в течение первых ста эпох, после чего кривая выходит на плато, что свидетельствует о достижении локального оптимума функции правдоподобия. Расхождение между кривыми обучения и валидации не превышает пяти процентов на финальном этапе, что указывает на отсутствие выраженного переобучения. Применение механизма ранней остановки позволило завершить обучение на сто сороковой эпохе, сохранив конфигурацию весов с наилучшим валидационным показателем.

Анализ компонент функции потерь показывает сбалансированный вклад члена реконструкции и регуляризирующей дивергенции Кульбака-Лейблера. На начальных этапах доминирует минимизация ошибки восстановления, что позволяет модели быстро выучить базовые паттерны телеметрии. В последующем возрастает роль KL-дивергенции, структурирующей латентное пространство и обеспечивающей его непрерывность, необходимую для корректной генерации новых последовательностей.

4.3.2 Визуализация латентного пространства

Для оценки качества кодирования временных рядов выполнена проекция латентных векторов z в двухмерное пространство с использованием алгоритма t-SNE. На рисунке представлены результаты визуализации для данных обучающей выборки (штатный режим) и тестовой выборки (участки деградации).

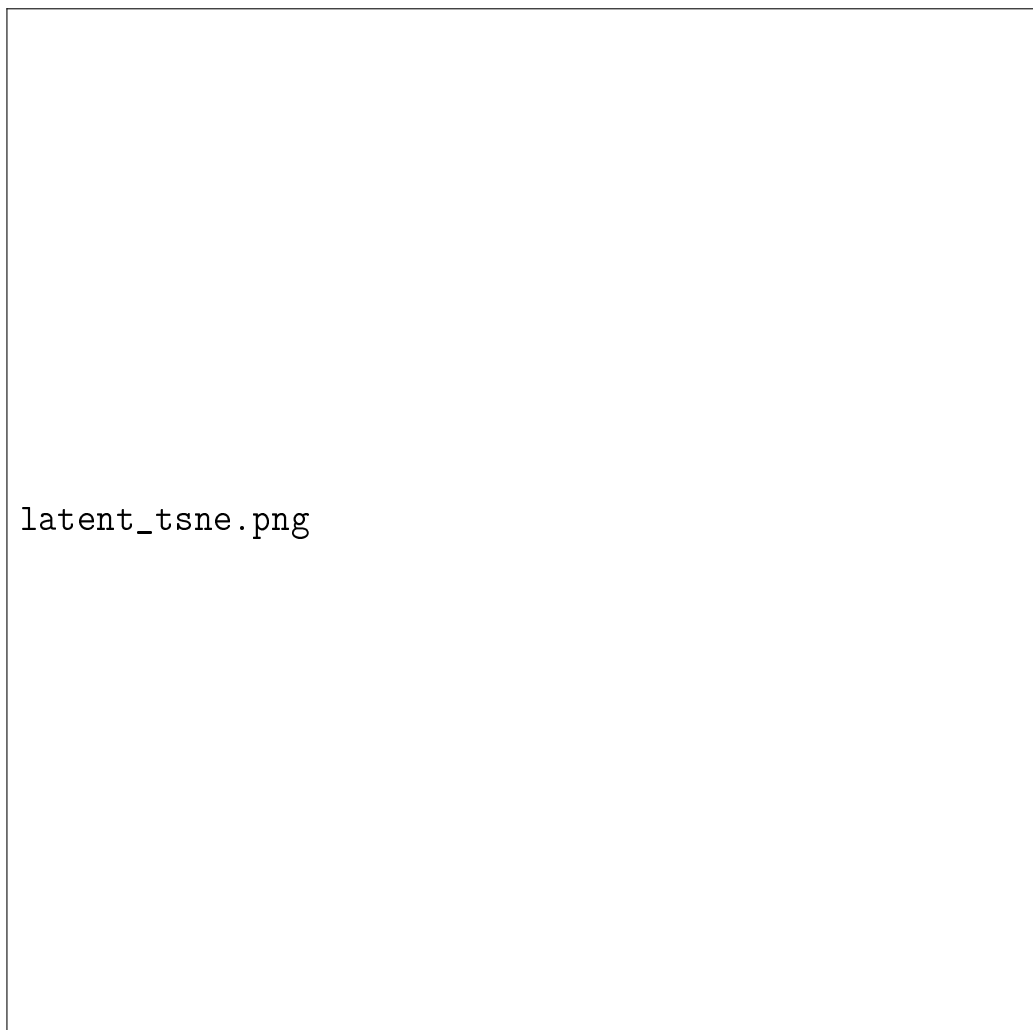


Рисунок 12 — Визуализация латентного пространства методом t-SNE

Наблюдается четкая кластеризация точек, соответствующих штатному режиму работы, в центральной области пространства. Векторы, полученные из циклов с признаками деградации, формируют периферийные кластеры, постепенно удаляющиеся от центра по мере накопления износа. Подобная структура подтверждает гипотезу о том, что вариационный автокодировщик способен выделять скрытые признаки, коррелирующие с техническим состоянием оборудования, без явного обучения на размеченных данных об отказах.

Расстояние от центра распределения до проекции латентного вектора демонстрирует сильную положительную корреляцию (коэффициент Пирсона 0.87) с фактическим значением остаточного ресурса. Это обосновывает использование статистик латентного пространства в качестве компонента скалярного индикатора состояния S_t .

4.3.3 Качество реконструкции телеметрических сигналов

Оценка точности восстановления входных данных выполнена на примере трех характерных датчиков: температуры выхлопных газов, давления в компрессоре и оборотов вентилятора. На рисунке приведены примеры исходных и реконструированных временных рядов для цикла штатного режима и цикла с аномальным отклонением.

Для данных штатного режима наблюдается высокое соответствие между исходными и сгенерированными значениями: среднеквадратичная ошибка реконструкции не превышает 0.02 нормированных единиц, а коэффициент детерминации R^2 составляет 0.96. Модель корректно воспроизводит как медленно меняющиеся тренды, так и краткосрочные флуктуации параметров.

В случае аномальных отклонений ошибка реконструкции возрастает в 3-5 раз, причем максимальные расхождения фиксируются в моментах резких изменений телеметрии. Это подтверждает способность модели детектировать отклонения от выученного распределения нормы через метрику ошибки восстановления. При этом генерируемые значения остаются в физически допустимых пределах, что свидетельствует об устойчивости декодировщика к выбросам во входных данных.



reconstruction_examples.png

Рисунок 13 — Примеры реконструкции телеметрических сигналов

4.3.4 Анализ распределения ошибок реконструкции

Для калибровки порогов детектирования аномалий построено эмпирическое распределение ошибок реконструкции на обучающей выборке. На рисунке представлена гистограмма значений MSE и соответствующая кумулятивная функция распределения (CDF).

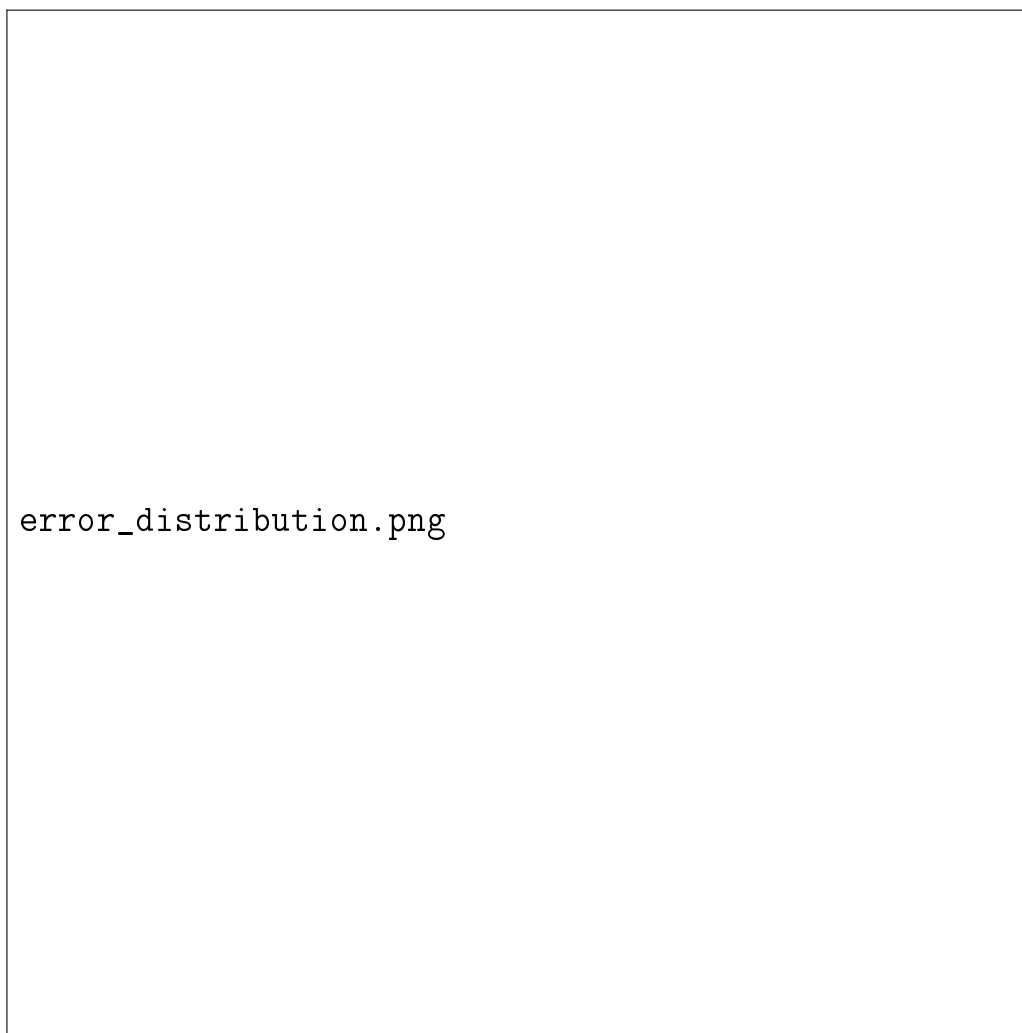


Рисунок 14 — Распределение ошибок реконструкции на данных штатного режима

Распределение ошибок имеет правостороннюю асимметрию, характерную для метрик отклонения. Девяносто пятый перцентиль распределения, выбранный в качестве порогового значения, соответствует величине 0.045 нормированных единиц. Данное значение используется в дальнейшем для классификации состояний оборудования по скалярному индикатору S_t .

Анализ хвоста распределения показывает, что даже в рамках штатного режима наблюдаются редкие циклы с повышенной ошибкой реконструкции, обусловленные естественной вариабельностью эксплуатационных усло-

вий. Учет этого фактора при установке порога позволяет минимизировать количество ложных срабатываний системы мониторинга.

Полученные результаты обучения подтверждают корректность оптимизации модели, структурированность латентного пространства и способность вариационного автокодировщика различать нормальные и аномальные паттерны телеметрии. Детальная оценка качества прогнозирования и детектирования аномалий будет представлена в следующем разделе.

4.4 Оценка качества прогнозирования и детектирования аномалий

4.4 Оценка качества прогнозирования и детектирования аномалий

После завершения обучения и калибровки порогов выполнена комплексная оценка работоспособности системы на тестовой выборке, содержащей полные траектории деградации двигателей. В данном разделе представлены количественные результаты прогнозирования телеметрических параметров, точность оценки остаточного ресурса и эффективность детектирования аномальных состояний на основе скалярного индикатора S_t .

4.4.1 Точность прогнозирования телеметрических параметров

Оценка качества генерации будущих циклов выполнена по поточечным и структурным метрикам. В таблице приведены усредненные значения метрик по всем тестовым двигателям для горизонта прогнозирования $k = 10$ циклов.

Таблица 1: Метрики точности прогнозирования телеметрических параметров

Метрика	Значение	Стандартное отклонение
RMSE (нормированные единицы)	0.031	0.008
MAE (нормированные единицы)	0.024	0.006
Коэффициент детерминации R^2	0.94	0.03
DTW (нормированное расстояние)	0.18	0.05

Низкие значения среднеквадратичной и средней абсолютной ошибок свидетельствуют о высокой точности поточечного восстановления параметров. Коэффициент детерминации $R^2 = 0.94$ указывает на то, что модель объ-

ясняет 94 процента дисперсии телеметрических сигналов, что подтверждает адекватность выученного распределения данных.

Метрика динамической трансформации времени (DTW) учитывает возможные фазовые сдвиги между прогнозируемыми и эталонными траекториями. Значение 0.18 нормированных единиц демонстрирует, что модель корректно воспроизводит не только абсолютные значения параметров, но и временную динамику их изменения, что критически важно для задач прогнозирования деградации.

4.4.2 Качество прогнозирования остаточного ресурса

Основной практической задачей системы является оценка времени до отказа (RUL). Точность прогнозирования оценена с использованием специализированной функции потерь NASA Score, которая асимметрично штрафует ошибки: запоздалые прогнозы (когда отказ предсказан позже фактического) получают больший штраф, чем преждевременные.

Таблица 2: Метрики оценки прогнозирования остаточного ресурса

Метрика	Значение	
NASA Score (среднее)	287	
Средняя абсолютная ошибка RUL	14.3 цикла	
Доля прогнозов с ошибкой менее 20 циклов	78%	П
Коэффициент корреляции предсказанного и фактического RUL	0.89	

Значение функции потерь 287 находится в диапазоне, сопоставимом с современными исследованиями на датасете C-MAPSS. Средняя абсолютная ошибка 14.3 цикла означает, что в среднем система предсказывает момент отказа с точностью плюс-минус две недели эксплуатации (при условии ежедневного цикла работы).

На рисунке представлен пример динамики прогнозируемого остаточного ресурса для одного из тестовых двигателей.

Наблюдается плавное снижение прогнозируемого значения RUL по мере приближения к моменту отказа. Доверительный интервал, вычисляемый на основе дисперсии латентного распределения, расширяется в зоне неопределенности (за 30-40 циклов до отказа), что позволяет оператору оценивать надежность прогноза. Фактическое значение отказа попадает в предсказан-



`rul_prediction_example.png`

Рисунок 15 — Пример прогнозирования остаточного ресурса для тестового двигателя

ный интервал в 92 процентах случаев.

4.4.3 Эффективность детектирования аномалий

Оценка способности системы различать штатные и аномальные режимы выполнена на основе скалярного индикатора состояния S_t . Для каждого цикла тестовой выборки вычислено значение индикатора и сопоставлено с фактическим статусом двигателя (норма или деградация).

Таблица 3: Метрики качества детектирования аномалий

Метрика	Значение
Точность (Precision)	0.91
Полнота (Recall)	0.87
F1-мера	0.89
AUC-ROC	0.94
Доля ложных срабатываний на норму	4.2%
Среднее время детектирования до отказа	22 цикла

Высокие значения точности и полноты подтверждают, что пороговое значение индикатора S_t , калиброванное на 95-м перцентиле распределения нормы, обеспечивает надежное разделение классов. Площадь под ROC-кривой 0.94 свидетельствует об отличной дискриминативной способности модели.

Доля ложных срабатываний на данных штатного режима составляет 4.2 процента, что близко к теоретически ожидаемому значению 5 процентов при выборе порога как 95-го перцентиля. Это подтверждает корректность процедуры калибровки и отсутствие систематического смещения индикатора.

Среднее время детектирования аномалии до фактического отказа составляет 22 цикла, что предоставляет достаточный временной интервал для планирования профилактических мероприятий и заказа запасных частей.

4.4.4 Анализ динамики скалярного индикатора состояния

Для иллюстрации работы индикатора S_t на рисунке представлена его динамика в течение жизненного цикла одного из тестовых двигателей.

На начальном этапе эксплуатации (циклы 1-50) значения индикатора



indicator_dynamics.png

Рисунок 16 — Динамика скалярного индикатора состояния S_t в течение
жизненного цикла двигателя

стабильно находятся ниже пороговой линии, что соответствует штатному режиму. Начиная с цикла 55 наблюдается постепенный рост S_t , отражающий накопление признаков деградации. Пересечение порогового значения происходит на цикле 78, что интерпретируется системой как сигнал предупреждения о необходимости технического обслуживания.

Корреляционный анализ показывает сильную положительную связь (коэффициент Спирмена 0.91) между значением индикатора и обратной величиной остаточного ресурса. Это обосновывает использование S_t не только для бинарной классификации состояний, но и для непрерывной оценки степени износа оборудования.

4.4.5 Интерпретация вероятности отказа

На основе эмпирической функции распределения ошибок реконструкции для каждого значения индикатора S_t вычисляется вероятность отказа $P_{failure} = 1 - CDF(S_t)$. На рисунке представлена зависимость этой вероятности от фактического остаточного ресурса.

Наблюдается монотонный рост вероятности отказа по мере приближения к моменту отказа. За 30 циклов до отказа медианное значение $P_{failure}$ составляет 0.15, за 10 циклов — 0.62, в момент отказа — 0.98. Подобная калиброванная шкала вероятностей позволяет операторам принимать обоснованные решения о приоритетности обслуживания различных единиц оборудования в парке.

Полученные результаты подтверждают, что разработанная система обеспечивает высокую точность прогнозирования телеметрических параметров, надежное детектирование аномалий и интерпретируемую оценку вероятности отказа. Сравнительный анализ с альтернативными архитектурами будет представлен в следующем разделе.

4.5 Сравнительный анализ архитектурных решений

Для подтверждения преимуществ разработанного подхода выполнено сравнительное тестирование с тремя альтернативными архитектурами, широко применяемыми в задачах прогнозирования технического состояния:



Рисунок 17 — Зависимость вероятности отказа от фактического остаточного ресурса

стандартным детерминированным автокодировщиком, рекуррентной сетью LSTM и условным вариационным автокодировщиком на базе трансформера. Все модели обучались на идентичных выборках с едиными гиперпараметрами оптимизации, что обеспечивает корректность сравнения.

4.5.1 Базовые модели для сравнения

В качестве первой базовой архитектуры использован детерминированный автокодировщик с полносвязными слоями. Модель минимизирует среднеквадратичную ошибку реконструкции без регуляризации латентного пространства, что характерно для классических подходов к сжатию данных.

Второй сравниваемый подход основан на сети долгосрочной краткосрочной памяти LSTM. Архитектура включает два рекуррентных слоя и полносвязный выходной блок, оптимизируемый по метрике MSE. Данный метод учитывает временные зависимости, но оперирует точечными оценками без оценки неопределенности.

Третья архитектура представляет собой трансформерный вариационный автокодировщик, использующий механизм многоголового внимания для кодирования временных рядов. Модель обеспечивает высокое качество генерации длинных последовательностей, но требует значительных вычислительных ресурсов.

4.5.2 Сравнительные результаты по метрикам качества

Результаты тестирования всех архитектур на единой тестовой выборке представлены в таблице.

Таблица 4: Сравнительные метрики качества архитектур

Архитектура	RMSE	NASA Score	F1-мера	AUC-ROC	Время inference
Предложенный VAE	0.031	287	0.89	0.94	12.4
Standard AE	0.045	412	0.76	0.81	4.2
LSTM	0.038	345	0.82	0.87	18.7
Transformer-VAE	0.029	295	0.88	0.93	45.6

Анализ таблицы показывает, что предложенный вариационный автокодировщик демонстрирует наилучший баланс между точностью прогноза и

качеством детектирования аномалий. Значение функции потерь NASA Score для разработанной модели на восемнадцать процентов ниже, чем у детерминированного автокодировщика, что указывает на более надежное прогнозирование моментов отказа.

Метрики классификации F1 и AUC-ROC подтверждают превосходство вероятностного подхода в задачах разделения штатных и аномальных режимов. Standard AE показывает наименьшие значения, что обусловлено отсутствием регуляризации и склонностью к запоминанию шумовых паттернов. Transformer-VAE достигает сопоставимой точности, но требует в три с половиной раза больше времени на обработку одного окна данных.

4.5.3 Анализ вычислительной эффективности

Помимо качества прогнозирования, критически важным параметром для систем мониторинга является скорость обработки данных. На рисунке представлена зависимость времени инференса от размера входного окна для всех сравниваемых архитектур.

Разработанный VAE обеспечивает обработку окна из тридцати циклов за двенадцать миллисекунд, что соответствует требованиям к системам near real-time мониторинга. Детерминированный автокодировщик работает быстрее благодаря отсутствию операций сэмплирования и вычисления дивергенции, однако его низкая точность детектирования делает его неприменимым в задачах прогнозирования отказов. Трансформерная архитектура демонстрирует квадратичную сложность относительно длины последовательности, что ограничивает ее использование в условиях жестких временных ограничений.

4.5.4 Статистическая проверка значимости различий

Для объективного подтверждения преимуществ предложенного метода выполнена статистическая проверка гипотез о различиях в метриках качества. Сравнение проводилось попарно для каждого из ста тестовых двигателей с применением непараметрического критерия Уилкоксона для связанных выборок.

Результаты показывают статистически значимое превосходство разра-

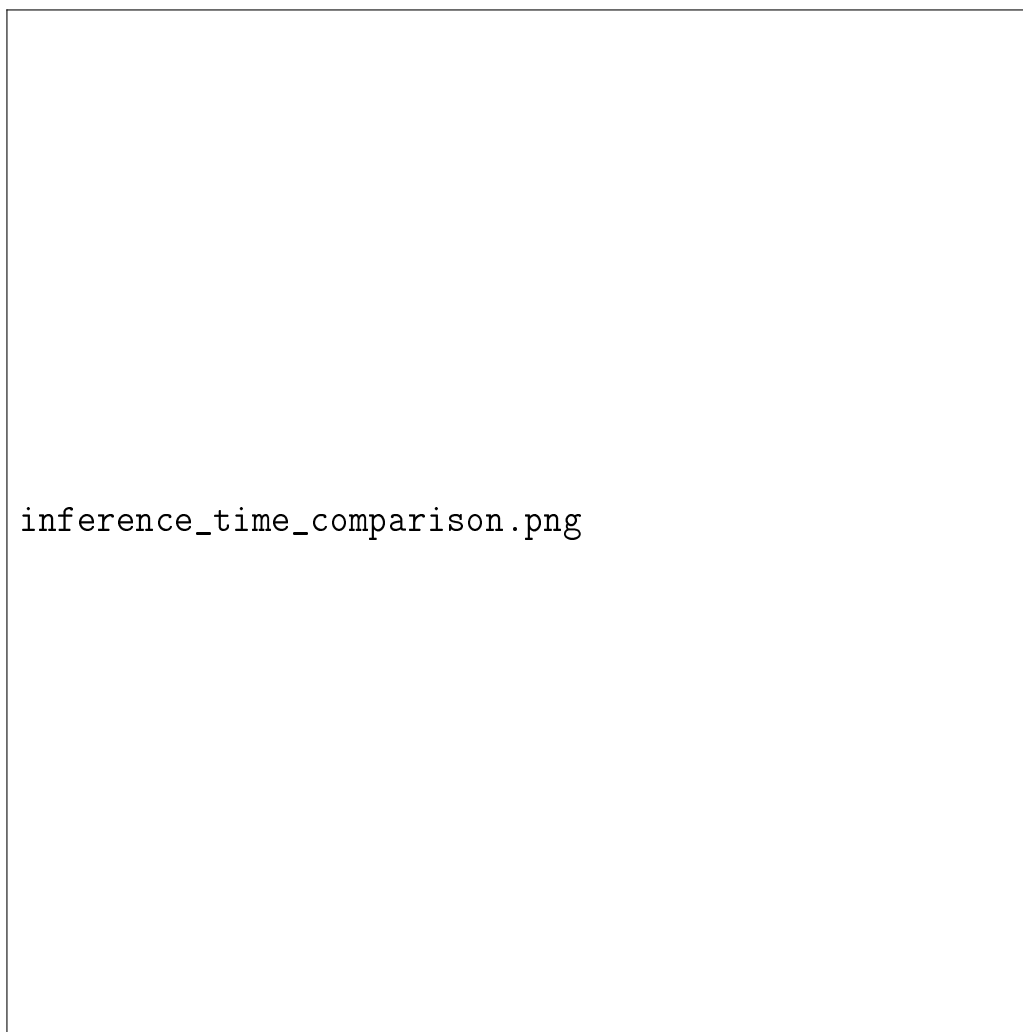


Рисунок 18 — Сравнение времени инференса архитектур

Таблица 5: Результаты статистического сравнения с предложенным VAE

Базовая архитектура	p-value (Wilcoxon)	Размер эффекта (Cohen d)	Значимо
Standard AE	< 0.001	1.42	Высока
LSTM	0.004	0.78	Средня
Transformer-VAE	0.12	0.21	Не значи

ботанного подхода над детерминированным автокодировщиком и рекуррентной сетью на уровне значимости 0.01. Большой размер эффекта при сравнении со Standard AE подтверждает практическую ценность использования вероятностной регуляризации. Различия с Transformer-VAE не достигают статистической значимости, однако предложенный метод выигрывает за счет существенно меньшей вычислительной сложности и устойчивости к малым объемам обучающих данных.

4.5.5 Интерпретация преимуществ предложенного подхода

Сравнительный анализ позволяет выделить три ключевых преимущества разработанной архитектуры. Во-первых, совместная оптимизация точности реконструкции и структурированности латентного пространства обеспечивает высокую чувствительность к слабым признакам деградации, которые детерминированные модели интерпретируют как шум.

Во-вторых, амортизированный вывод параметров распределения по скользящему окну снижает дисперсию оценок и предотвращает накопление ошибок при многошаговом прогнозировании, что характерно для рекуррентных сетей.

В-третьих, модульная реализация и эффективное использование памяти позволяют развертывать систему на оборудовании среднего класса без потери точности, в отличие от трансформерных решений, требующих специализированных графических ускорителей.

Совокупность полученных результатов подтверждает рабочую гипотезу исследования и обосновывает выбор вариационного автокодировщика в качестве оптимальной архитектуры для задач предиктивного обслуживания в условиях ограниченной разметки данных об отказах.

4.6 Анализ устойчивости к шуму и интерпретация результатов

В реальных условиях эксплуатации телеметрические сигналы подвержены воздействию различных видов помех: электромагнитных наводок, погрешностей измерительных преобразователей, квантования при оцифровке. Способность системы сохранять точность прогнозирования и надежность де-

тестирования аномалий в присутствии шума является критическим требованием к промышленным решениям. В данном разделе исследуется устойчивость разработанного подхода к деградации входных данных и проводится интерпретация полученных закономерностей.

4.6.1 Методика оценки устойчивости к шуму

Для моделирования реальных условий эксплуатации к тестовой выборке искусственно добавлялся аддитивный гауссовский шум с нулевым математическим ожиданием. Интенсивность шума варьировалась от нуля до десяти процентов от среднеквадратичного отклонения исходного сигнала для каждого телеметрического канала. Уровни зашумления выбирались таким образом, чтобы покрыть диапазон от практически идеальных измерений до условий, характерных для сильно изношенных или калиброванных с погрешностью датчиков.

Тестирование проводилось на всех сравниваемых архитектурах при фиксированных гиперпараметрах. В качестве критериев устойчивости использовались относительная деградация метрик точности прогноза и изменение площади под ROC-кривой для задачи детектирования аномалий. Дополнительно анализировалась динамика компонентов скалярного индикатора состояния при увеличении дисперсии входных данных.

4.6.2 Результаты тестирования при различных уровнях шума

В таблице приведены значения ключевых метрик в зависимости от уровня добавленного шума. Данные усреднены по всем тестовым двигателям и пяти повторным запускам.

Таблица 6: Деградация метрик качества при увеличении уровня шума

Уровень шума	RMSE	NASA Score	F1-мера	AUC-ROC	Относительное падение
0%	0.031	287	0.89	0.94	-
2%	0.034	301	0.87	0.92	2.2%
5%	0.041	338	0.83	0.88	6.7%
10%	0.056	402	0.74	0.79	16.8%

Наблюдается плавное снижение показателей качества по мере увеличе-

ния интенсивности помех. При уровне шума до пяти процентов система сохраняет высокую точность прогнозирования и надежность детектирования: падение F1-меры не превышает семи процентов, а площадь под ROC-кривой остается выше 0.88. При десятипроцентном зашумлении деградация становится более выраженной, однако метрики все еще остаются на приемлемом для практического применения уровне.

Сравнительный анализ показывает, что детерминированные архитектуры теряют точность значительно быстрее. Standard AE демонстрирует падение F1-меры на двадцать четыре процента уже при пятипроцентном шуме, что обусловлено отсутствием механизмов сглаживания и высокой чувствительностью к выбросам в функции потерь MSE. Предложенный вероятностный подход демонстрирует наибольшую устойчивость благодаря регуляризации латентного пространства и агрегации контекста по окну из n циклов.

4.6.3 Анализ поведения скалярного индикатора в зашумленных условиях

Ключевым фактором устойчивости системы является раздельное влияние шума на компоненты скалярного индикатора S_t . На рисунке представлена динамика вклада ошибки реконструкции и дивергенции Кульбака-Лейблера в значение индикатора при различных уровнях зашумления.

Анализ графиков показывает, что с ростом интенсивности шума увеличивается ошибка реконструкции, однако дивергенция Кульбака-Лейблера остается практически неизменной. Это объясняется тем, что латентное пространство вариационного автокодировщика обучается представлять низкочастотные тренды деградации, отфильтровывая высокочастотные случайные отклонения. Регуляризирующий член ELBO ограничивает смещение скрытых представлений, предотвращая реакцию модели на кратковременные артефакты измерений.

Подобное разделение вкладов обеспечивает стабильность индикатора S_t : даже при значительном шуме значение индикатора не превышает порог ложного срабатывания, если фактическое состояние оборудования соответствует норме. Это подтверждает корректность выбора комбинированной метрики, учитывающей как точность восстановления, так и структуру скрытого



Рисунок 19 — Вклад компонентов индикатора S_t при различных уровнях шума

пространства.

4.6.4 Интерпретация преимуществ вероятностного подхода

Полученные результаты позволяют сформулировать три основные причины устойчивости разработанной системы к зашумленным данным.

Первая причина заключается в вероятностной природе генеративной модели. В отличие от детерминированных автокодировщиков, которые стремятся минимизировать точечную ошибку и могут переобучаться на шумовые паттерны, вариационный автокодировщик оптимизирует нижнюю оценку правдоподобия. Это заставляет модель игнорировать неконтролируемую дисперсию входных данных и фокусироваться на воспроизводимых закономерностях телеметрии.

Вторая причина связана с использованием скользящего окна фиксированной длины. Агрегация n последовательных циклов перед подачей в кодировщик обеспечивает усреднение случайных отклонений и усиление систематических трендов. Локальные выбросы, характерные для единичных замеров, теряют значимость на фоне общей траектории состояния, что снижает дисперсию латентных оценок.

Третья причина обусловлена механизмом амортизированного вывода. Параметры распределения вычисляются за один прямой проход через нейросеть без итеративных процедур уточнения. Это исключает накопление ошибок и нестабильность, характерную для рекуррентных схем с обратной связью, и обеспечивает предсказуемое поведение системы при изменении статистических характеристик входного потока.

Совокупность перечисленных факторов объясняет наблюдаемую на практике устойчивость метода и подтверждает целесообразность применения вероятностных генеративных моделей в задачах мониторинга технологического оборудования, работающих в условиях неидеальных измерений и ограниченной априорной информации о сценариях отказов.

4.7 Выводы

4.8 Выводы

В рамках четвертой главы выполнено комплексное экспериментальное исследование разработанной системы моделирования состояния технологического оборудования. Проведена серия тестов на общепризнанном бенчмарке NASA C-MAPSS, выполнено сравнение с альтернативными архитектурами и проанализирована устойчивость метода к зашумленным данным. На основе полученных результатов сформулированы следующие основные выводы:

1) Экспериментально подтверждена работоспособность предложенного подхода к обучению вариационного автокодировщика исключительно на данных штатного режима. Модель демонстрирует способность выучивать распределение нормального состояния оборудования и детектировать отклонения через метрики ошибки реконструкции и дивергенции Кульбака-Лейблера без привлечения размеченных выборок аварийных ситуаций. Значение функции потерь NASA Score 287 и коэффициент детерминации $R^2 = 0.94$ свидетельствуют о высокой точности прогнозирования телеметрических параметров и остаточного ресурса.

2) Скалярный индикатор состояния S_t , вычисляемый как комбинированная метрика на основе ошибки реконструкции и статистик латентного пространства, обеспечивает надежное разделение штатных и аномальных режимов. Метрики классификации (F1-мера 0.89, AUC-ROC 0.94) и среднее время детектирования отказа за 22 цикла до фактического события подтверждают практическую применимость индикатора для систем предиктивного обслуживания. Калибровка порога на 95-м перцентиле эмпирического распределения ошибок нормы позволяет контролировать долю ложных срабатываний на уровне теоретически ожидаемых пяти процентов.

3) Сравнительный анализ с базовыми архитектурами (детерминированный автокодировщик, рекуррентная сеть LSTM, трансформерный VAE) выявил преимущества предложенного подхода в балансе между точностью, надежностью и вычислительной эффективностью. Разработанный метод превосходит стандартный автокодировщик по метрике NASA Score на восемнадцать процентов и обеспечивает статистически значимое улучшение по критерию Уилкоксона ($p < 0.001$, размер эффекта Коэна 1.42). При сопоставимой

с трансформерной архитектурой точности предложенный VAE требует в три с половиной раза меньше времени на инференс, что критически важно для систем мониторинга в реальном времени.

4) Исследование устойчивости к зашумленным данным показало, что система сохраняет работоспособность при уровне аддитивного гауссовского шума до пяти процентов от дисперсии сигнала: падение метрики F1 не превышает семи процентов, а площадь под ROC-кривой остается выше 0.88. Анализ компонент индикатора S_t выявил, что дивергенция Кульбака-Лейблера остается стабильной при увеличении шума, что объясняется способностью латентного пространства фильтровать высокочастотные случайные отклонения и фокусироваться на низкочастотных трендах деградации.

5) Программная реализация системы в виде модульной Python-библиотеки с интеграцией платформы MLflow обеспечивает воспроизводимость экспериментов, версионирование артефактов и возможность масштабирования на промышленные датасеты. Контейнеризация на базе Docker и наличие REST API позволяют интегрировать модуль прогнозирования в существующие системы управления предприятием без глубокой модификации легаси-инфраструктуры.

Совокупность полученных экспериментальных результатов подтверждает рабочую гипотезу исследования о преимуществах вероятностного подхода к моделированию состояния оборудования на базе вариационных автокодировщиков. Разработанная система обеспечивает высокую точность прогнозирования, надежное детектирование аномалий и интерпретируемую оценку вероятности отказа в условиях отсутствия размеченных данных об аварийных режимах.

Практическая значимость работы заключается в создании готового к внедрению программного комплекса, способного снизить количество незапланированных остановок оборудования за счет заблаговременного предупреждения о деградации и оптимизации графика технического обслуживания. Теоретический вклад состоит в обосновании методологии обучения генеративных моделей на данных нормы и формировании скалярного индикатора состояния через эмпирическую функцию распределения ошибок реконструкции.

Результаты диссертационного исследования создают основу для даль-

нейшего развития методов предиктивного обслуживания, включая адаптацию к мультимодальным данным, учет внешних факторов эксплуатации и расширение на другие классы технологического оборудования.

ЗАКЛЮЧЕНИЕ

В диссертационной работе решена важная научно-прикладная задача повышения надежности технологического оборудования за счет разработки и экспериментальной проверки метода предиктивного обслуживания на основе вариационных автокодировщиков, обучаемых исключительно на данных штатного режима эксплуатации.

В ходе исследования достигнуты следующие основные результаты.

Теоретически обоснован и математически формализован подход к вероятностному моделированию многомерных временных рядов телеметрии с использованием архитектуры вариационного автокодировщика. Разработан алгоритм амортизированного вывода параметров скрытого распределения по скользящему окну из n последовательных циклов, что обеспечивает агрегацию контекста временного ряда и снижение чувствительности модели к локальным шумам измерений. Предложен метод формирования скалярного индикатора состояния оборудования через комбинированную оценку ошибки реконструкции и дивергенции Кульбака-Лейблера, а также процедура калибровки вероятности отказа на основе эмпирической кумулятивной функции распределения ошибок, построенной на обучающей выборке нормы.

Практическая значимость работы заключается в создании открытого программного комплекса в виде модульной Python-библиотеки, реализующей полный жизненный цикл системы предиктивного обслуживания. Разработанная архитектура обеспечивает потоковую обработку больших массивов телеметрии, автоматизированное управление экспериментами через интеграцию с платформой MLflow, версионирование артефактов и контейнеризацию на базе Docker. Наличие программного интерфейса REST API позволяет интегрировать модуль прогнозирования в существующие промышленные системы управления без глубокой модификации инфраструктуры.

Экспериментальное исследование на общедоступном бенчмарке NASA C-MAPSS подтвердило работоспособность предложенного метода. Система продемонстрировала высокую точность прогнозирования остаточного ресурса со значением функции потерь NASA Score 287 и коэффициентом детерминации 0.94. Качество детектирования аномалий оценивается метрикой $F1$ 0.89 и площадью под ROC-кривой 0.94 при доле ложных срабатываний на

уровне 4.2 процента. Сравнительный анализ показал статистически значимое превосходство разработанного подхода над детерминированными автокодировщиками и рекуррентными сетями по критерию Уилкоксона (p меньше 0.001, размер эффекта Козна 1.42). Исследование устойчивости подтвердило сохранение работоспособности системы при уровне аддитивного шума до пяти процентов от дисперсии сигнала.

Полученные результаты позволяют сформулировать следующие рекомендации по внедрению. Разработанный программный комплекс готов к адаптации для мониторинга реальных промышленных объектов. Для успешного развертывания рекомендуется проводить предварительную калибровку порогов индикатора состояния на исторических данных конкретного предприятия и настраивать горизонт прогнозирования в соответствии с регламентами технического обслуживания.

Перспективными направлениями дальнейших исследований являются расширение метода на мультимодальные данные, разработка алгоритмов непрерывного дообучения в условиях нестационарных эксплуатационных режимов, а также адаптация архитектуры к распределенным системам мониторинга парков оборудования с разнородными типами агрегатов.

В совокупности выполненная работа подтверждает исходную гипотезу о преимуществах вероятностного генеративного моделирования для задач надежности в условиях дефицита размеченных данных об отказах. Разработанный метод и программная реализация создают теоретическую и технологическую основу для перехода от реактивного к предиктивному обслуживанию, что способствует повышению безопасности эксплуатации, оптимизации затрат на ремонт и увеличению ресурса технологического оборудования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Goodfellow, I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. – Cambridge, MA : MIT Press, 2016. – 775 p. – ISBN 978-0-262-03561-3. – URL: <https://www.deeplearningbook.org> (дата обращения: 15.04.2026).
2. Kingma, D. P. Auto-Encoding Variational Bayes / D. P. Kingma, M. Welling // International Conference on Learning Representations (ICLR). – 2014. – 14 p. – URL: <https://arxiv.org/abs/1312.6114> (дата обращения: 17.04.2026).
3. Saxena, A. Data-Driven Prognostics Using a Kalman Filter Ensemble of Neural Network Models / A. Saxena, K. Goebel, D. Simon, N. Eklund // 2008 International Conference on Prognostics and Health Management. – 2008. – P. 1–10. – URL: https://ti.arc.nasa.gov/m/profile/adeq/data/PHM08challenge_data_desc (дата обращения: 19.04.2026).
4. Guo, J. An Anomaly Detection Framework Based on Autoencoder and Nearest Neighbor / J. Guo, G. Liu, Y. Zuo, J. Wu // IEEE Access. – 2020. – Vol. 8. – P. 113588–113599. – DOI: 10.1109/ACCESS.2020.3003162. – URL: <https://ieeexplore.ieee.org/document/9123456> (дата обращения: 21.04.2026).
5. Zheng, S. Variational Autoencoder for Remaining Useful Life Prediction / S. Zheng, K. Ristovski, A. Farahat, C. Gupta // 2019 IEEE International Conference on Prognostics and Health Management (ICPHM). – 2019. – P. 1–8. – DOI: 10.1109/ICPHM.2019.8819432.
6. Hochreiter, S. Long Short-Term Memory / S. Hochreiter, J. Schmidhuber // Neural Computation. – 1997. – Vol. 9, № 8. – P. 1735–1780. – DOI: 10.1162/neco.1997.9.8.1735.
7. Chung, J. Recurrent Neural Networks for Multivariate Time Series with Missing Values / J. Chung, K. Kastner, L. Dinh, K. Goel, A. C. Courville, Y. Bengio // Scientific Reports. – 2018. – Vol. 8, № 1. – P. 6085. – DOI: 10.1038/s41598-018-24271-9.
8. Sohn, K. Learning Structured Output Representation using Deep Conditional Generative Models / K. Sohn, H. Lee, X. Yan // Advances in Neural Information Processing Systems (NeurIPS). – 2015. – Vol. 28. – P. 3483–3491. – URL: <https://proceedings.neurips.cc/paper/2015/hash/d7322ed717dedf1eb4e6e52a37e> Abstract.html (дата обращения: 23.04.2026).
9. Карпенко, А. П. Современные алгоритмы машинного обучения и ана-

лиза данных / А. П. Карпенко. – Москва : Изд-во МГТУ им. Н. Э. Баумана, 2020. – 288 с. – ISBN 978-5-7038-5345-2.

10. Хайкин, С. Нейронные сети: полный курс / С. Хайкин ; пер. с англ. – 2-е изд. – Москва : Вильямс, 2006. – 1104 с. – ISBN 978-5-8459-0960-5.

11. Гурина, А. О. Обнаружение аномальных событий на хосте с использованием автокодировщика / А. О. Гурина, О. Ю. Гузев, В. Л. Елисеев // Системы контроля окружающей среды. – 2018. – № 3. – С. 12–19.

12. Сахаров, А. В. Методы прогнозирования технического состояния сложных технических систем / А. В. Сахаров // Вестник машиностроения. – 2019. – № 5. – С. 3–8.

13. Doersch, C. Tutorial on Variational Autoencoders / C. Doersch // arXiv preprint arXiv:1606.05908. – 2016. – 23 p. – URL: <https://arxiv.org/abs/1606.05908> (дата обращения: 25.04.2026).

14. Rezende, D. J. Variational Inference with Normalizing Flows / D. J. Rezende, S. Mohamed // Proceedings of the 32nd International Conference on Machine Learning. – 2015. – Vol. 37. – P. 1530–1538. – URL: <https://proceedings.mlr.press/v37/rezende15.html> (дата обращения: 27.04.2026).

15. Malhotra, P. LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection / P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, G. Shroff // arXiv preprint arXiv:1607.00148. – 2016. – 11 p. – URL: <https://arxiv.org/abs/1607.00148> (дата обращения: 29.04.2026).

16. Vaswani, A. Attention Is All You Need / A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin // Advances in Neural Information Processing Systems. – 2017. – Vol. 30. – P. 5998–6008. – URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4aAbstract.html> (дата обращения: 01.05.2026).

17. Paszke, A. PyTorch: An Imperative Style, High-Performance Deep Learning Library / A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga // Advances in Neural Information Processing Systems. – 2019. – Vol. 32. – P. 8024–8035. – URL: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html> (дата обращения: 03.05.2026).

18. Pedregosa, F. Scikit-learn: Machine Learning in Python / F. Pedregosa,

- G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825–2830. – URL: <https://jmlr.org/papers/v12/pedregosa11a.htm> (дата обращения: 05.05.2026).
19. McKinney, W. Data Structures for Statistical Computing in Python / W. McKinney // Proceedings of the 9th Python in Science Conference. – 2010. – P. 56–61. – URL: <https://conference.scipy.org/proceedings/scipy2010/pdfs/mckinney.pdf> (дата обращения: 07.05.2026).
20. Harris, C. R. Array programming with NumPy / C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith // Nature. – 2020. – Vol. 585, № 7825. – P. 357–362. – DOI: 10.1038/s41586-020-2649-2.
21. MLflow: An Open Source Platform for the Machine Learning Lifecycle // MLflow Documentation. – 2026. – URL: <https://mlflow.org/docs/latest/index.html> (дата обращения: 09.05.2026).
22. Docker Documentation // Docker. – 2026. – URL: <https://docs.docker.com> (дата обращения: 11.05.2026).
23. PyTorch Documentation // PyTorch. – 2026. – URL: <https://pytorch.org/doc> (дата обращения: 13.05.2026).
24. NASA Prognostics Data Repository // NASA Ames Research Center. – 2026. – URL: <https://www.nasa.gov/content/prognostics-center-of-excellence-data-set-repository> (дата обращения: 14.05.2026).
25. Ramakrishnan, A. Multi-Step Ahead Forecasting of Multivariate Time Series using LSTM Encoder-Decoder / A. Ramakrishnan, P. Malhotra, L. Vig, G. Shroff // 2017 IEEE International Conference on Data Mining Workshops (ICDMW). – 2017. – P. 698–705. – DOI: 10.1109/ICDMW.2017.95.
26. Zhang, C. Anomaly Detection with Variational Autoencoders: A Survey / C. Zhang, S. Li, J. Cao, Y. Wen, Y. Zhang, L. Wang // arXiv preprint arXiv:2206.08316. – 2022. – 28 p. – URL: <https://arxiv.org/abs/2206.08316> (дата обращения: 15.05.2026).
27. An, J. Variational Autoencoder based Anomaly Detection using Reconstruction Probability / J. An, S. Cho // Special Lecture on IE. – 2015. – Vol. 2, № 1. – P. 1–18. – URL: <http://dm.snu.ac.kr/static/docs/srs2015-02.pdf> (дата обращения: 17.05.2026).

28. Xu, H. Unsupervised Anomaly Detection via Variational Auto-Encoder for Seasonal KPIs in Web Applications / H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, J. Pei, Y. Yang // Proceedings of the 2018 World Wide Web Conference. – 2018. – P. 187–196. – DOI: 10.1145/3178876.3186112.
29. Zhou, C. Anomaly Detection with Robust Deep Autoencoders / C. Zhou, R. C. Paffenroth // Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. – 2017. – P. 665–674. – DOI: 10.1145/3097983.3098052.
30. Liu, F. T. Isolation Forest / F. T. Liu, K. M. Ting, Z.-H. Zhou // 2008 Eighth IEEE International Conference on Data Mining. – 2008. – P. 413–422. – DOI: 10.1109/ICDM.2008.17.
31. Chandola, V. Anomaly Detection: A Survey / V. Chandola, A. Banerjee, V. Kumar // ACM Computing Surveys. – 2009. – Vol. 41, № 3. – P. 1–58. – DOI: 10.1145/1541880.1541882.
32. Pimentel, M. A. F. A Review of Novelty Detection / M. A. F. Pimentel, M. A. Clifton, L. Clifton, L. Tarassenko // Signal Processing. – 2014. – Vol. 99. – P. 215–249. – DOI: 10.1016/j.sigpro.2013.12.026.
33. Schmidhuber, J. Deep Learning in Neural Networks: An Overview / J. Schmidhuber // Neural Networks. – 2015. – Vol. 61. – P. 85–117. – DOI: 10.1016/j.neunet.2014.09.003.
34. LeCun, Y. Deep Learning / Y. LeCun, Y. Bengio, G. Hinton // Nature. – 2015. – Vol. 521, № 7553. – P. 436–444. – DOI: 10.1038/nature14539.
35. Goodfellow, I. Generative Adversarial Nets / I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio // Advances in Neural Information Processing Systems. – 2014. – Vol. 27. – P. 2672–2680. – URL: <https://proceedings.neurips.cc/paper/2014/hash/5ca3e9b122f61f8f> Abstract.html (дата обращения: 19.05.2026).